

Dossier technique

Dispositif de suivi de la fistule artério-veineuse : mesurer et documenter le thrill, sans comprimer la fistule

Encadrement	**Pr. ZAZ**
Version	**10 novembre 2026**
Contenu	**21 lots de travail, une annexe sur les preuves**
Nature	**Prototype sur appareil virtuel, données synthétiques**

Aide à la décision uniquement : la décision d'un examen écho-Doppler reste médicale.

Table des matières

	Avant-propos, lecture du dossier et glossaire	02

PARTIE 1 · CADRAGE, MÉTHODE ET OUTILS

LT-01	Cadrage et pré-développement terminé	03
LT-02	Environnement d'ingénierie multi-agents terminé	06
LT-03	Cartographie des connaissances du projet terminé	09
LT-04	Architecture et squelette logiciel (monorepo) terminé	12
LT-05	Développement piloté par les tests et traçabilité terminé (cadre)	17
LT-06	Salle de contrôle visuelle terminé	20
LT-07	Système de design et génération des interfaces en cours	24
LT-08	Pack de contexte et documentation académique terminé	27

PARTIE 2 · RÉALISATION DU SYSTÈME

LT-09	Contrat d'API et modèle de données (jalon M1) terminé	30
LT-10	Moteur de traitement du signal (jalon M2) terminé	34
LT-11	Cœur backend et pipeline de mesure en cours	37
LT-12	Application web et système de design terminé	40
LT-13	Banc de formation et concordance en cours	43
LT-14	Intégration et démonstration de bout en bout terminé	46

PARTIE 3 · INTERFACES, SÉCURITÉ, IA ET MATÉRIEL

LT-15	Interfaces et graphiques médicaux terminé (finitions en cours)	50
LT-16	Droits, authentification et durcissement terminé	53
LT-17	Couche d'IA explicable terminé (chemin vers les données réelles : voir §8)	56
LT-18	Protocole appareil et squelette de firmware terminé sur PC ; jamais exécuté sur une carte	59
LT-19	Kit d'étude sur banc et étude virtuelle terminé (étude virtuelle réalisée sur données synthétiques)	62

PARTIE 4 · REPRISE DES ÉCRANS ET SYNTHÈSE

LT-20	Intégration, limites et suite terminé (chiffres à jour au 2026-11-10)	65
LT-21	Refonte des écrans avec les compétences de conception, et vérification dans un vrai navigateur terminé (finitions en §8)	68
Annexe	Ce qui est prouvé, et ce qui ne l'est pas	73

AVANT-PROPOS

Lire ce dossier

Dispositif de Suivi de la Fistule Artério-Veineuse, outil d'aide à la décision pour la surveillance de l'abord vasculaire des patients hémodialysés. Encadrement : Pr. ZAZ.

> Aide à la décision uniquement : la décision d'un examen écho-Doppler reste médicale.

Comment lire ce dossier

Chaque **lot de travail** (LT) documente une étape réalisée : le problème traité, la méthode, ce qui a été construit, comment cela a été vérifié, les décisions prises et leurs limites. Chaque chiffre renvoie à sa source (fichier du dépôt ou commande exécutée). Les versions PDF se trouvent dans `pdf/`.

Glossaire

TERME	DÉFINITION SIMPLE
FAV	Fistule artério-veineuse : raccordement chirurgical d'une artère et d'une veine du bras, utilisé pour la dialyse.
Thrill	Vibration perceptible au toucher au-dessus de la FAV, causée par le flux sanguin turbulent.
Baseline	Référence personnelle d'un patient, calculée sur ses 3 premières mesures valides.
RMS	Moyenne quadratique : mesure de l'énergie du signal de vibration, en g.
FFT / STFT	Transformées de Fourier : décomposent le signal en fréquences (globalement / au cours du temps).
Porte qualité	Ensemble de règles qui accepte ou rejette une mesure (appui, mouvement, durée propre).
Monorepo	Un seul dépôt de code contenant plusieurs applications qui partagent des outils et des types.
TDD	Développement piloté par les tests : on écrit le test qui échoue avant le code qui le fait passer.
C4	Méthode de diagrammes d'architecture en 4 niveaux : contexte, conteneurs, composants, code.
Orchestrateur	Session d'IA principale qui planifie, délègue à d'autres agents, vérifie et intègre.
UC	Cas d'utilisation du cahier des charges (UC1 à UC13).
FP / FC	Fonctions principales / fonctions contraintes du cahier des charges.

PARTIE 1

Cadrage, méthode et outils

Le besoin, l'environnement d'ingénierie, l'architecture et la démarche de test.

LOT DE TRAVAIL 1

Cadrage et pré-développement

Jalon Avant M0 | terminé | 2026-11-03

Résumé

Le cahier des charges DSFAV a été transformé en un dossier de pré-développement en 8 phases (charte, besoins, architecture, UX, données et API, sécurité, plan d'exécution, validation). Une relecture croisée a mis en évidence **15 contradictions** entre documents, chacune tranchée dans une spécification canonique, et un plan de construction en **10 jalons logiciels (M0 à M9) et 4 jalons matériels (H0 à H3)** a été établi (`pre-dev/09-canonical-spec.md` §1, `pre-dev/10-build-plan.md` §2-3).

1. Contexte et problématique

Les FAV présentent environ **23 % d'échec primaire** et **60 % de perméabilité à un an** (`pre-dev/00-project-charter.md` §1). La surveillance de routine repose sur la palpation, subjective ; l'écho-Doppler, référence, est trop coûteux pour être répété à chaque séance (3 fois par semaine). Le besoin : une mesure **objective, rapide (20 à 30 s), non invasive et peu coûteuse**, comparée à la référence personnelle du patient.

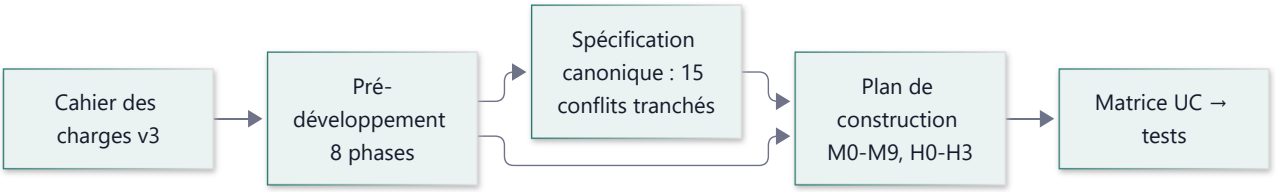
2. Objectifs

- Standardiser l'acquisition en 20 à 30 s par patient (FP1).
- Coût matériel du prototype 900 à 1 500 MAD, coût amorti 0,50 à 1,00 MAD par mesure (FC2).
- Autonomie d'au moins 20 mesures par jour (FC3).
- Banc d'essai pédagogique pour les internes (FP4). Source : `pre-dev/00-project-charter.md` §2.

3. Démarche et méthode

Le document source unifié mélangeait trois natures de contenu : documents de phase, prompts de design et extraits de code. Ils ont été séparés (`SOURCE-RECONCILIATION.md`) en `pre-dev/` (phases), `artifacts/` (design) et `artifacts/implementation-reference/` (code de référence, non destiné à la production). Les identifiants présents dans la source n'ont pas été recopiés.

4. Conception



5. Réalisation

- 8 phases documentées dans `pre-dev/` (état dans `pre-dev/PROGRESS.md`).
- Spécification canonique `pre-dev/09-canonical-spec.md` : zones de pression uniques (< 2,5 / 2,5-3,0 / 3,0-8,5 / 8,5-9,0 / > 9,0 kPa), une seule mention légale, cycle de vie de la baseline (verrouillée après 3 mesures valides), score d'écart en z-score diagonal, rôles et permissions.
- Plan `pre-dev/10-build-plan.md` : jalons, tests « golden » G1 à G9 pour le traitement du signal, **15 défauts** relevés dans le code de référence (§7).

6. Vérification et résultats

ÉLÉMENT	RÉSULTAT	SOURCE
Phases complètes	0, 2, 3, 4, 5, 7 complètes ; 1 partielle ; 6 et 8 en brouillon	`pre-dev/PROGRESS.md`
Conflits identifiés et tranchés	15	`09-canonical-spec.md` §1
Décisions ouvertes pour le superviseur	7	`09-canonical-spec.md` §13

7. Décisions et justifications

- **Baseline sur 3 mesures** plutôt que « première mesure » : une seule mesure ne donne pas d'écart-type (ADR-003, canonique §4).
- **Seuil d'écart par défaut 3,4** au lieu de 2,5 : avec 3 caractéristiques, 2,5 déclencherait environ 10 % de fausses alertes sur des séances normales (loi du khi-deux à 3 degrés de liberté) ; 3,4 vise environ 1 % (canonique §4).

8. Limites et points ouverts

Étude de marché et analyse concurrentielle non étayées (phase 1 partielle). Les valeurs « à définir » du cahier (seuil, tolérance inter-appareils, temps de recharge) sont des valeurs par défaut à calibrer au banc (M9), pas des faits cliniques.

9. Ce qu'il faut retenir

- Le projet a une source de vérité unique pour chaque règle : la spécification canonique.
- Chaque cas d'utilisation sera prouvé par un test nommé.
- DSFAV ne pose jamais de diagnostic.

10. Questions possibles du jury

- **Pourquoi comparer le patient à lui-même et non à une norme ?** Les FAV diffèrent beaucoup entre patients ; une dérive personnelle est plus informative qu'un seuil de population (ADR-003).
- **Pourquoi 3 mesures pour la baseline ?** Minimum pour estimer une moyenne et une dispersion ; un plancher de 5 % de la moyenne évite qu'une baseline trop stable gonfle le score (canonique §4).
- **Que se passe-t-il si les documents se contredisent ?** La spécification canonique tranche ; le cahier reste l'autorité (AGENTS.md §2).

Références

`DSFAV_cahier_des_charges_FINAL_v3.pdf` · `pre-dev/00-project-charter.md` · `pre-dev/09-canonical-spec.md` · `pre-dev/10-build-plan.md` · `pre-dev/03-ADRs/`

LOT DE TRAVAIL 2

Environnement d'ingénierie multi-agents

Jalon M0 | terminé | 2026-11-07

Résumé

Nous avons installé un environnement où une session d'IA principale (l'orchestrateur) planifie le travail, le confie à des agents exécutants, fait relire chaque résultat par un agent vérificateur et n'intègre que le travail accepté. Le kit a passé **10 auto-tests sur 11**, **7 exécutants sur 10** ont répondu au test de disponibilité, et les règles propres à DSFAV ont été formalisées en **6 compétences** et **3 agents** (sources en §6).

1. Contexte et problématique

Un projet qui couvre firmware, traitement du signal, backend et interface demande beaucoup de travail répétitif mais exigeant en rigueur clinique. Confier ce travail à des agents d'IA n'est utile que si chaque contribution est **cadrée** (périmètre de fichiers), **vérifiée** (tests et règles) et **tracée** (journal).

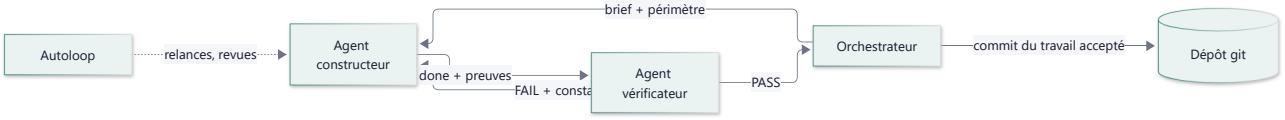
2. Objectifs

- Un hub de coordination fonctionnel dans le dépôt (critère : `hub.mjs status` répond).
- Au moins 2 exécutants disponibles (critère : `dispatch.mjs ping all`).
- Des règles cliniques non négociables lisibles par tout agent avant tout travail.

3. Démarche et méthode

1. Installation du kit `agent-hub-orchestrator` v2.2.0 avec sauvegarde de l'ancienne copie, puis auto-test.
2. Initialisation du hub dans le dépôt (`.agent-hub/`) avec le pack « generic » : le pack « ui-migration » vise la copie de sites web et ne convient pas à un dispositif médical (décision D-004).
3. Test de disponibilité de chaque exécutant **avant** toute planification (règle du kit : ne jamais confier une tâche à un exécutant non testé).
4. Rédaction des règles DSFAV, des compétences et des agents.

4. Conception



Chaque agent lit d'abord `AGENTS.md` (règles), puis son brief, puis la compétence de son domaine. Le vérificateur est en lecture seule et applique une « règle d'arrêt » : après un premier tour de reprise, seul un défaut fonctionnel, clinique, d'accessibilité ou visuel majeur peut faire échouer une tâche.

5. Réalisation

- Règles : `AGENTS.md` §4 (10 règles cliniques : mention légale unique, aucune donnée nominative, zones de pression, porte qualité, baseline, seuils configurables, statuts, aucune perte de session, règle du banc, aucune conclusion clinique sur données synthétiques) et §5 (8 règles d'ingénierie).
- Compétences : `.claude/skills/dsfav-context` (domaine et chiffres canoniques), `dsfav-tdd`, `dsfav-dsp`, `dsfav-backend`, `dsfav-web`, `dsfav-academic`.
- Agents : `.claude/agents/dsfav-builder.md`, `dsfav-verifier.md`, `dsfav-dsp-tester.md`.
- Portes qualité : `docs/GATES.md` (G1 types, G2 lint, G3 tests écrits d'abord, G4 règles cliniques, G5 contrat, G6 secrets, G7 périmètre).
- Mémoire du projet : `docs/DECISIONS.md`, `docs/BACKLOG.md`, `docs/ARCHITECTURE.md`.

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Auto-test du kit	10 PASS, 1 FAIL (`keeper.mjs --help` lance le service au lieu d'afficher l'aide)	sortie de `engine/selftest.mjs`, 2026-11-07
Exécutants disponibles	7/10 : Claude Sonnet 4.6, Gemini 3.8 Flash (×2), GLM, DeepSeek flash, GPT-6 (codex), Grok 4.7 (cursor)	`dispatch.mjs ping all`
Exécutants indisponibles	3 (claude-ds1/2/3, crédit DeepSeek épuisé)	idem
Compétences DSFAV indexées	5 au premier index, 6 avec `dsfav-academic`	`tools/skills.mjs build`

7. Décisions et justifications

D-004 (pack générique et compétences propres au projet). Python est appelé via `uv` car la commande `python` du poste pointe vers un alias du Microsoft Store (D-002).

8. Limites et points ouverts

Le défaut `keeper.mjs --help` est à corriger dans le kit maître (hors périmètre du projet). Aucune tâche n'a encore suivi le cycle complet construction → revue → acceptation → commit par le hub : ce sera la première tâche du jalon M1.

9. Ce qu'il faut retenir

- L'IA produit, mais un vérificateur indépendant et des tests décident.
- Les règles cliniques sont écrites une fois et lues par tous les agents.
- On ne planifie que sur des exécutants testés.

10. Questions possibles du jury

- **Comment garantir que l'IA ne viole pas une règle clinique ?** Règles explicites (AGENTS.md §4), contrôles automatiques (GATES G4), agent vérificateur séparé du constructeur.
- **Pourquoi plusieurs modèles d'IA ?** Disponibilité et coût : si un fournisseur est saturé, la tâche passe au suivant ; la revue par un modèle différent réduit les biais communs.
- **Qui est responsable du code final ?** L'orchestrateur n'intègre que le travail accepté ; l'équipe humaine valide les jalons.

Références

`AGENTS.md` · `docs/GATES.md` · `docs/DECISIONS.md` · `.claude/skills/` · `.claude/agents/` · `.agent-hub/PROTOCOL.md`

LOT DE TRAVAIL 3

Cartographie des connaissances du projet

Jalon M0 | terminé | 2026-11-07

Résumé

Les 46 documents du projet (cahier des charges, dossier de pré-développement, design, contrat d'API) ont été convertis en un **graphe de connaissances de 284 concepts et 739 relations**, regroupés automatiquement en **11 communautés** thématiques. Le graphe sert de carte navigable du projet et révèle les concepts « ponts » et les zones faiblement documentées (`graphify-out/GRAPH_REPORT.md`).

---

1. Contexte et problématique

Le projet compte plus de 17 000 mots de spécification répartis en dizaines de fichiers. Il est difficile de voir quels documents parlent du même concept (par exemple UC7, la baseline) et où se trouvent les incohérences.

---

2. Objectifs

Obtenir une représentation visuelle et interrogeable des relations entre exigences, cas d'utilisation, composants et décisions, avec une traçabilité honnête de chaque relation (trouvée dans le texte ou déduite).

---

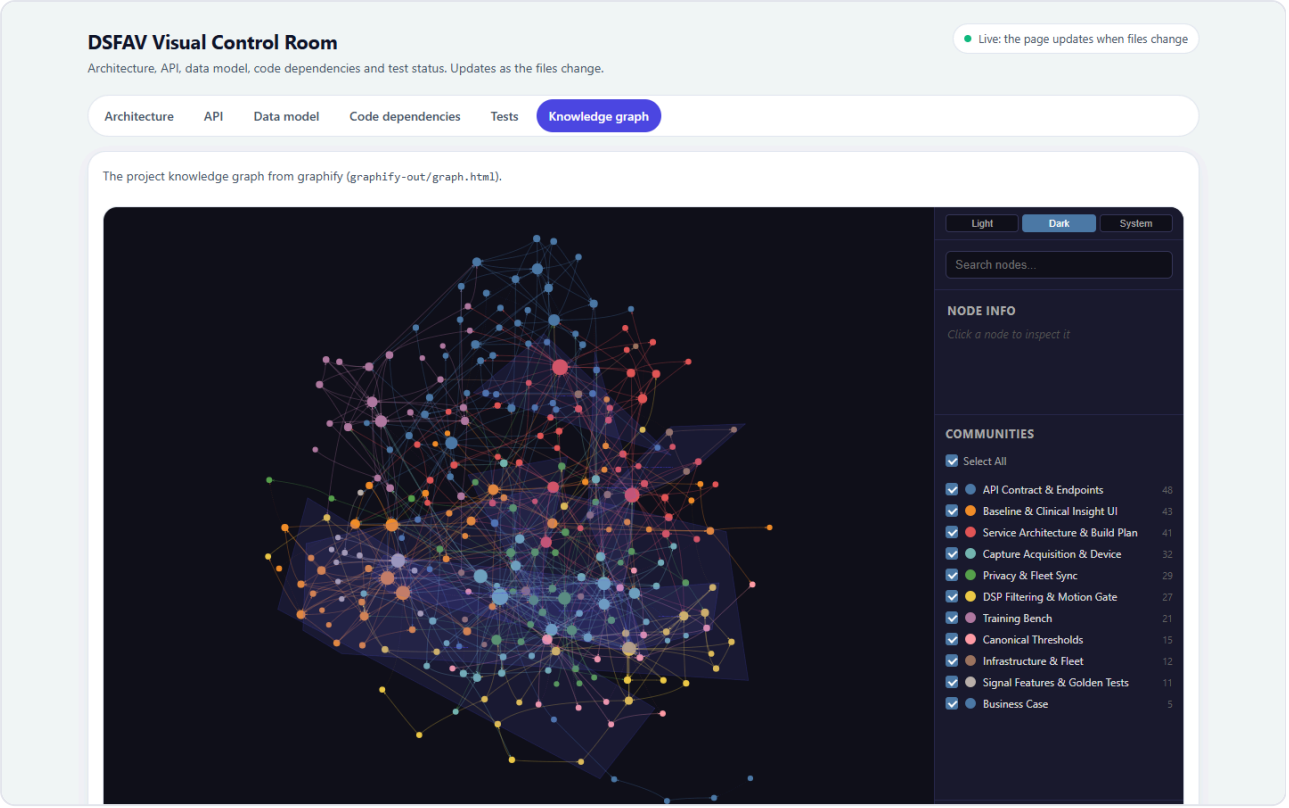
3. Démarche et méthode

Outil : graphify. Les fichiers techniques du kit et le document source brut ont été exclus (`.graphifyignore` , D-005) pour ne garder que le contenu DSFAV.

1. Détection : 46 fichiers, environ 23 300 mots (1 code, 44 documents, 1 PDF).
2. Extraction sémantique en parallèle par 3 agents légers (Claude Haiku), un par tiers du corpus ; extraction structurelle (AST) du contrat OpenAPI.
3. Fusion : les concepts partagés (UC1 à UC13, FP, FC, services) portent le même identifiant dans tous les lots, ce qui relie les documents entre eux (63 fusions).
4. Détection de communautés (algorithme de Louvain), nommage, rapport et page interactive.

Chaque relation est étiquetée **EXTRACTED** (explicite dans le texte), **INFERRED** (déduite, avec un score de confiance) ou **AMBIGUOUS** (incertaine, à vérifier).

4. Conception



5. Réalisation

- `graphify-out/graph.html` : graphe interactif (recherche, filtres par communauté).
- `graphify-out/GRAPH_REPORT.md` : rapport (concepts centraux, connexions surprenantes, questions suggérées).
- `graphify-out/graph.json` : données réutilisables.
- Le graphe est aussi intégré à la salle de contrôle visuelle (LT-06, onglet « Knowledge graph »).

6. Vérification et résultats

MESURE	VALEUR	SOURCE
Concepts / relations / communautés	284 / 739 / 11	sortie de la construction du graphe, 2026-11-07
Concept le plus connecté	UC1 Acquisition du thrill (26 relations)	`GRAPH_REPORT.md`, God Nodes
Suivants	API NestJS (25), unité ESP32-C3 (23), UC4, UC6, UC9 (21 chacun)	idem
Cycles d'import	aucun	idem
Coût d'extraction	484 217 jetons d'entrée (3 agents)	rapports d'usage des agents

Communautés : Contrat d'API · Baseline et interface clinique · Architecture et plan · Acquisition et appareil · Confidentialité et synchronisation · Filtrage et porte mouvement · Banc d'essai · Seuils canoniques · Infrastructure · Caractéristiques du signal et tests golden · Business case.

7. Décisions et justifications

D-005 : exclusion du kit et de la source brute, sinon le graphe aurait été dominé par l'outillage et par des doublons.

8. Limites et points ouverts

Les relations INFERRED et AMBIGUOUS sont produites par un modèle et doivent être vérifiées avant toute décision. Le rapport signale **17 nœuds faiblement connectés**, dont les rôles Infirmier, Interne et Administrateur : le tableau des rôles (canonique §10) devrait être relié plus explicitement aux cas d'utilisation. Le graphe se met à jour automatiquement pour le code (crochet git) ; les documents demandent une mise à jour manuelle (`/graphify . --update`).

9. Ce qu'il faut retenir

- UC1 est le pont entre presque toutes les parties du projet : c'est le flux le plus critique.
- Chaque relation dit si elle est trouvée ou supposée.
- Le graphe montre aussi ce qui manque.

10. Questions possibles du jury

- **Le graphe peut-il inventer des liens ?** Il peut en déduire ; ils sont marqués INFERRED avec un score, jamais présentés comme des faits.
- **À quoi sert-il concrètement ?** Retrouver tous les documents touchés par un changement (par exemple modifier la porte de pression), et repérer les zones peu spécifiées.

Références

`graphify-out/GRAPH_REPORT.md` · `.graphifyignore` · `docs/DECISIONS.md` D-005

LOT DE TRAVAIL 4

Architecture et squelette logiciel (monorepo)

Jalon M0 | terminé | 2026-11-07

Résumé

Le jalon M0 a produit le squelette exécutable de DSFAV : trois applications (interface web Next.js, API NestJS, service de traitement du signal Python), un paquet de types généré depuis le contrat d'API, la base de données décrite par Prisma, la composition Docker et l'intégration continue. **Vérification de types, analyse statique et tests passent pour les 8 paquets** ; l'interface web compile avec ses 9 écrans prévus (sorties des commandes du 2026-11-07, §6).

1. Contexte et problématique

Le plan impose « une tranche verticale à la fois » et « le contrat d'abord » (`pre-dev/10-build-plan.md` §1). Il faut donc un socle où chaque couche démarre, expose un point de santé et peut être testée, avant toute fonctionnalité clinique.

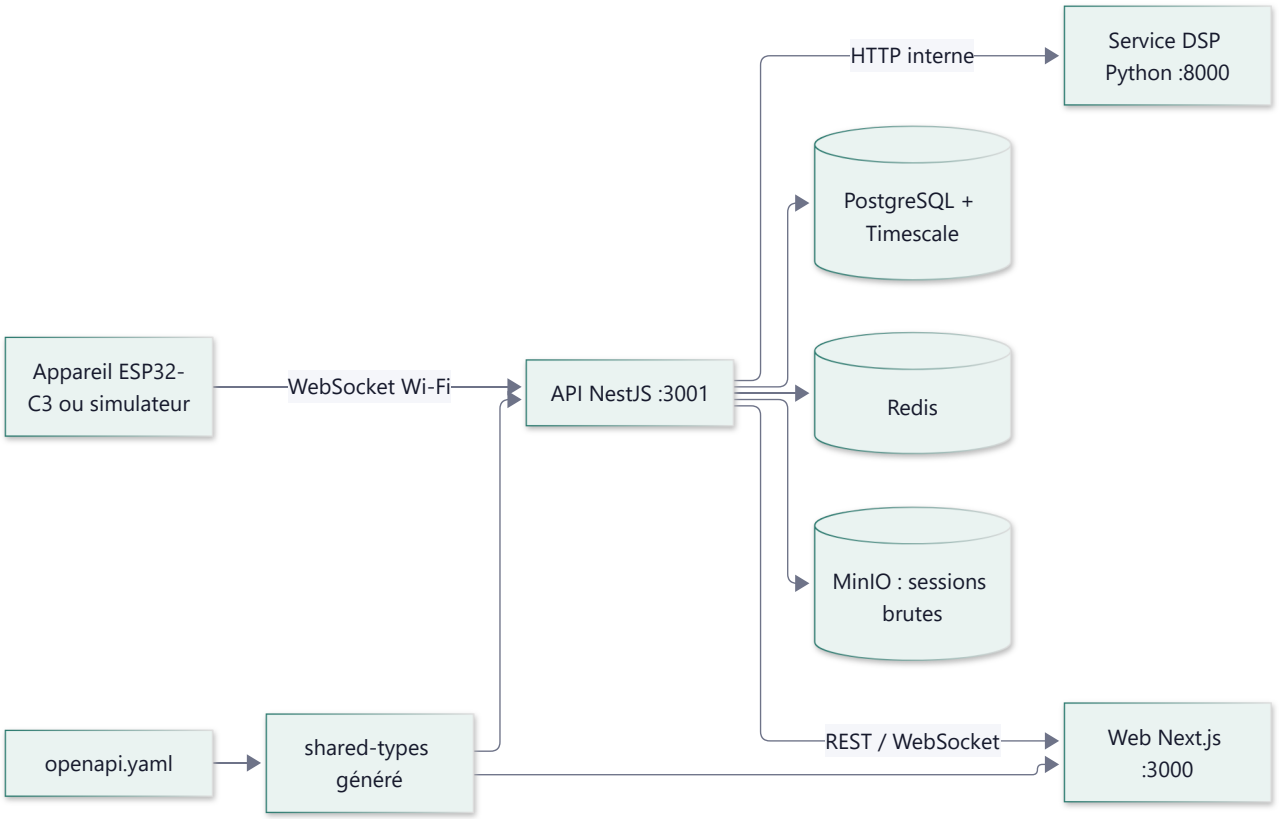
2. Objectifs

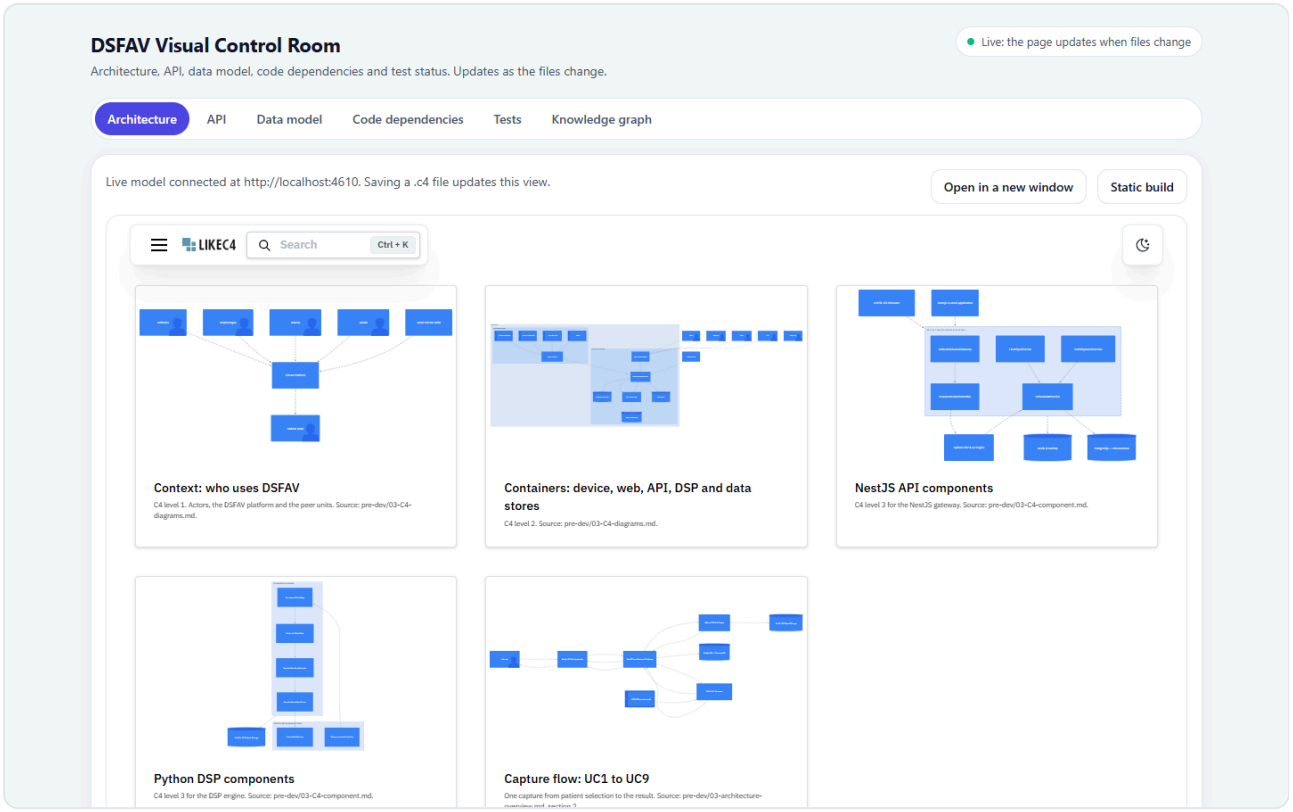
Critère du jalon M0 : un clone propre démarre la pile avec une commande et l'intégration continue est verte sur une suite de tests minimale (`10-build-plan.md` §3 M0).

3. Démarche et méthode

Le squelette a été généré par un agent exécutant (Claude Haiku) à partir d'un cahier précis (arborescence, ports, points de santé, tests, conteneurs), puis **vérifié indépendamment** par l'orchestrateur, qui a corrigé un écart (voir §7).

4. Conception





5. Réalisation

DOSSIER	RÔLE	POINT DE SANTÉ
`apps/web-nextjs`	9 écrans : connexion, tableau de bord, acquisition, patients, dossier patient, banc, flotte, paramètres	`/api/health`
`apps/api-nestjs`	API, schéma Prisma (`prisma/schema.prisma`), modules prévus (auth, patients, mesures, baseline, appareils, banc, audit, configuration)	`/health`
`apps/dsp-python`	bibliothèque pure `dsfav_dsp` + enveloppe FastAPI	`/health`
`packages/shared-types`	types TypeScript générés depuis `pre-dev/05-api-contracts/openapi.yaml`	n/a
`e2e/`	tests de bout en bout Playwright	n/a
`firmware/` , `scripts/simulator/`	emplacements des jalons H1-H3 et M4	n/a
`docker-compose.yml` (+ `.override.yml`)	6 services avec contrôles de santé ; aucun port de base de données publié hors développement	n/a
`.github/workflows/ci.yml`	lint, types, tests, contrat, composition	n/a

La mention légale est définie **une seule fois** (`apps/web-nextjs/src/libs/clinical/disclaimer.ts`) et importée par le composant d'affichage ; un test vérifie le texte exact.

6. Vérification et résultats

COMMANDE	RÉSULTAT	SOURCE
`pnpm install`	8 projets, à jour	sortie du 2026-11-07
`pnpm -r typecheck`	PASS (code 0)	idem
`pnpm -r lint`	PASS (ESLint, oxlint, Ruff)	idem
`pnpm -r test`	API 2/2, web 1/1, DSP 1/1, e2e 1 ignoré (pas de serveur)	`reports/junit/*.xml`
`pnpm --filter @dsfav/web build`	PASS, routes générées	sortie du build
`prisma validate`	PASS après correction	sortie de Prisma 7.10.1
Lint du contrat OpenAPI	0 erreur, 5 avertissements	rapport de l'agent, `redocly lint`
`docker compose config`	PASS (images non construites)	rapport de l'agent
Contrôles G4 / G6	aucune donnée nominative dans le schéma et le contrat ; aucun secret	grep du 2026-11-07

Versions : Next.js 16.4.5, React 19.3.0, NestJS 12.4.0, Prisma 7.10.1, FastAPI 0.162.1, NumPy 2.5.4, SciPy 1.18.1 (rapport de l'agent).

7. Décisions et justifications

- D-001 Next.js 16 au lieu de 15 ; D-002 Python 3.12 via `uv` ; D-003 chaque application garde son lanceur de tests natif, unifiés par des rapports JUnit.
- **Prisma 7** n'accepte plus l'URL de connexion dans le schéma ; elle a été déplacée dans `apps/api-nestjs/prisma.config.ts`. Le schéma reste la copie de référence du pré-développement, à amender au jalon M1.
- Prisma épinglé en 7.10.1 : l'étiquette « latest » du registre pointait vers une version candidate 8.0.

8. Limites et points ouverts

Les Dockerfiles ne sont pas encore construits ni testés. Le schéma Prisma n'intègre pas encore les ajouts de la spécification canonique (§6 : source de mesure, versions de baseline, audit, configuration versionnée) : c'est l'objet de M1. Les écrans ne contiennent qu'un titre et la mention légale.

9. Ce qu'il faut retenir

- Trois services séparés : interface, orchestration, calcul scientifique (ADR-001).
- Les types circulent dans un seul sens : contrat → types générés → applications.
- Un squelette vérifié vaut mieux que des fonctionnalités non testées.

10. Questions possibles du jury

- **Pourquoi Python pour le signal et non Node.js ?** NumPy et SciPy fournissent des filtres et transformées éprouvés ; NestJS orchestre l'authentification, la persistance et le temps réel (ADR-001).
- **Pourquoi un monorepo ?** Un seul contrat d'API partagé, une seule intégration continue, des changements atomiques entre API et interface.
- **Comment éviter d'exposer la base de données ?** Le fichier de composition de base ne publie aucun port de base ; seul le fichier de développement le fait.

Références

`pre-dev/07-execution-blueprint.md` · `pre-dev/03-ADRs/ADR-001-*.md` · `docs/ARCHITECTURE.md` · `docs/DECISIONS.md`

LOT DE TRAVAIL 5

Développement piloté par les tests et traçabilité

Jalon M0 | terminé (cadre) | 2026-11-07

Résumé

Nous avons mis en place un cadre où chaque cas d'utilisation du cahier des charges est relié à un test nommé, et où l'état de ces tests est visible dans une matrice de traçabilité qui se met à jour automatiquement. La matrice compte **26 lignes** (13 cas d'utilisation, 9 tests golden du signal, 4 propriétés transverses) ; à ce jour **1 est prouvée (mention légale), 0 en échec, 25 à écrire** (`docs/visual/traceability.json`, 2026-11-07).

---

1. Contexte et problématique

Dans un dispositif d'aide à la décision médicale, « l'écran existe » ne prouve rien. Le plan pose la règle : **un cas d'utilisation est terminé quand son test passe dans l'intégration continue** (`pre-dev/10-build-plan.md` §1.5).

---

2. Objectifs

- Une convention unique pour relier un test à une exigence.
- Une vue d'ensemble lisible sans lire le code.

---

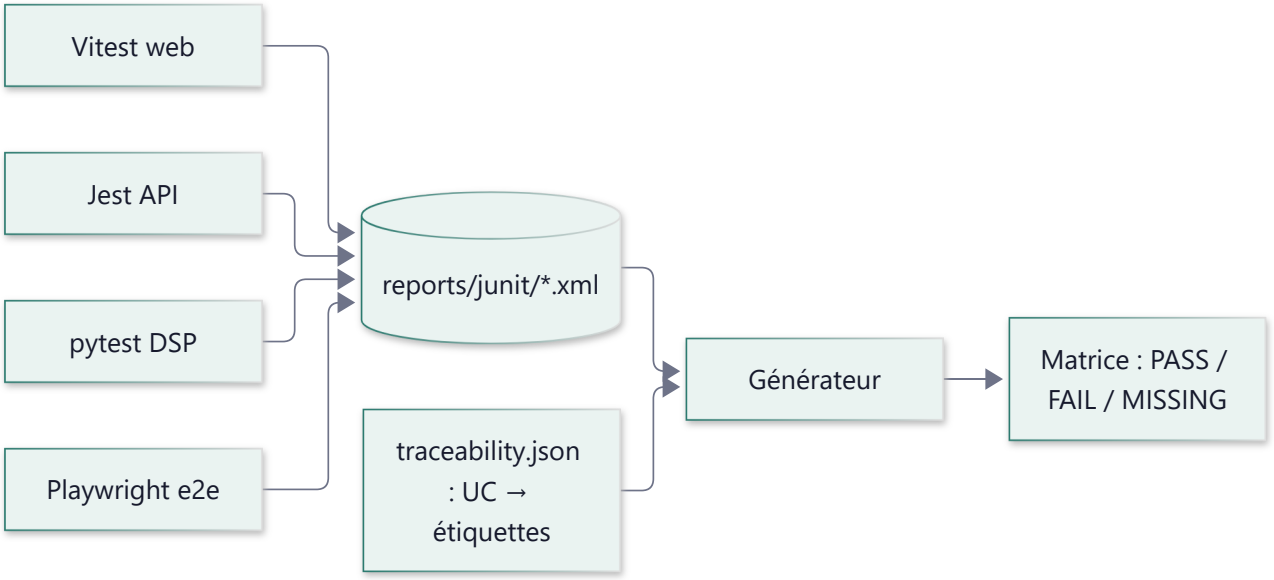
3. Démarche et méthode

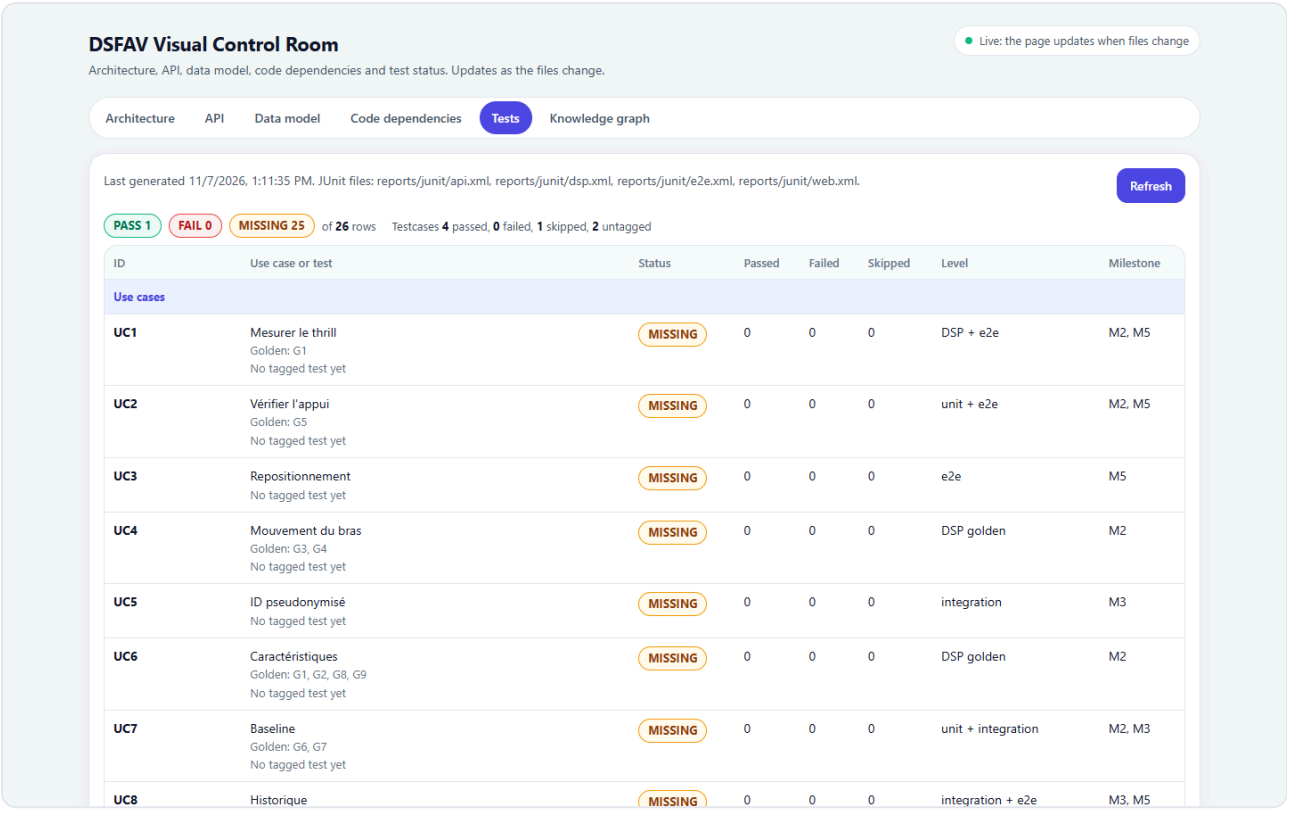
Cycle **rouge → vert → refactorisation** : écrire le test qui échoue pour la bonne raison, écrire le minimum qui le fait passer, nettoyer sans changer le comportement (`.claude/skills/dsfav-tdd/SKILL.md`).

Convention d'étiquettes dans le nom des tests : `[UC1]` … `[UC13]`, `[G1]` … `[G9]`, `[DISCLAIMER]`, `[NO-LOSS]`, `[PSEUDO-SCAN]`, `[AUTHZ-MATRIX]`. Exemple réel : `[DISCLAIMER] renders the exact clinical disclaimer` (`apps/web-nextjs/src/components/shared/disclaimer.test.tsx`).

Chaque lanceur écrit un rapport JUnit dans `reports/junit/` ; un générateur lit ces rapports et calcule l'état de chaque ligne.

4. Conception





5. Réalisation

- Lanceurs : Vitest (web), Jest (API), pytest + Hypothesis (DSP), Playwright (bout en bout).
- Matrice source : `tools/visual/traceability.json` (reprend `10-build-plan.md` §4).

- Générateur : `tools/visual/gen/traceability.mjs`.
- Tests golden du signal (G1 à G9) spécifiés avec leurs tolérances, et un générateur de signaux synthétiques dont les caractéristiques sont connues à l'avance (`10-build-plan.md` §5) ; ils seront écrits **avant** le pipeline par l'agent `dsfav-dsp-tester`.

6. Vérification et résultats

LIGNE	ÉTAT	SOURCE
DISCLAIMER	PASS	`reports/junit/web.xml`
HEALTH (API, DSP)	PASS	`reports/junit/api.xml`, `dsp.xml`
UC1 à UC13, G1 à G9, NO-LOSS, PSEUDO-SCAN, AUTHZ-MATRIX	MISSING (attendu au jalon M0)	`docs/visual/traceability.json`

7. Décisions et justifications

D-003 : chaque application garde son lanceur natif, et le format JUnit commun permet une vue unique.

8. Limites et points ouverts

La matrice prouve qu'un test existe et passe, pas qu'il est pertinent : la revue par l'agent vérificateur contrôle qu'il échouerait sans le changement. Les tests golden prouvent les mathématiques, pas la valeur clinique (AGENTS.md §4.10).

9. Ce qu'il faut retenir

- Une exigence sans test n'est pas terminée.
- Les limites sont testées des deux côtés et à l'égalité (G7 : l'égalité au seuil n'est pas une alerte).
- La matrice rend l'avancement visible pour un non-développeur.

10. Questions possibles du jury

- **Pourquoi des signaux synthétiques ?** On connaît la réponse attendue (RMS, rapport, fréquence), donc on peut vérifier le calcul exactement ; aucune donnée patient dans le dépôt.

- **Comment savoir que l'application ne perd aucune mesure ?** Test de propriété « no-loss » : avec coupures et doublons aléatoires, le nombre de sessions stockées égale le nombre de captures acceptées (`10-build-plan.md` §4).

Références

`pre-dev/10-build-plan.md` §4-5 · `.claude/skills/dsfav-tdd/SKILL.md` · `tools/visual/traceability.json`

LOT DE TRAVAIL 6

Salle de contrôle visuelle

Jalon M0 | terminé | 2026-11-07

Résumé

Une application locale réunit en six onglets les vues du projet : architecture C4 vivante, référence de l'API, modèle de données, dépendances du code, matrice des tests et graphe de connaissances. Elle se met à jour quand les fichiers changent. Les six onglets s'affichent **sans erreur de console** dans un navigateur réel (vérification Playwright/Edge du 2026-11-07).

---

1. Contexte et problématique

Le porteur du projet doit piloter et présenter le système sans lire le code. Les diagrammes dessinés à la main se périment dès que le code évolue.

---

2. Objectifs

Des vues **générées depuis les sources** (modèle C4, contrat d'API, schéma de base, imports du code, rapports de tests), toujours à jour, accessibles par une commande : `pnpm visual` .

---

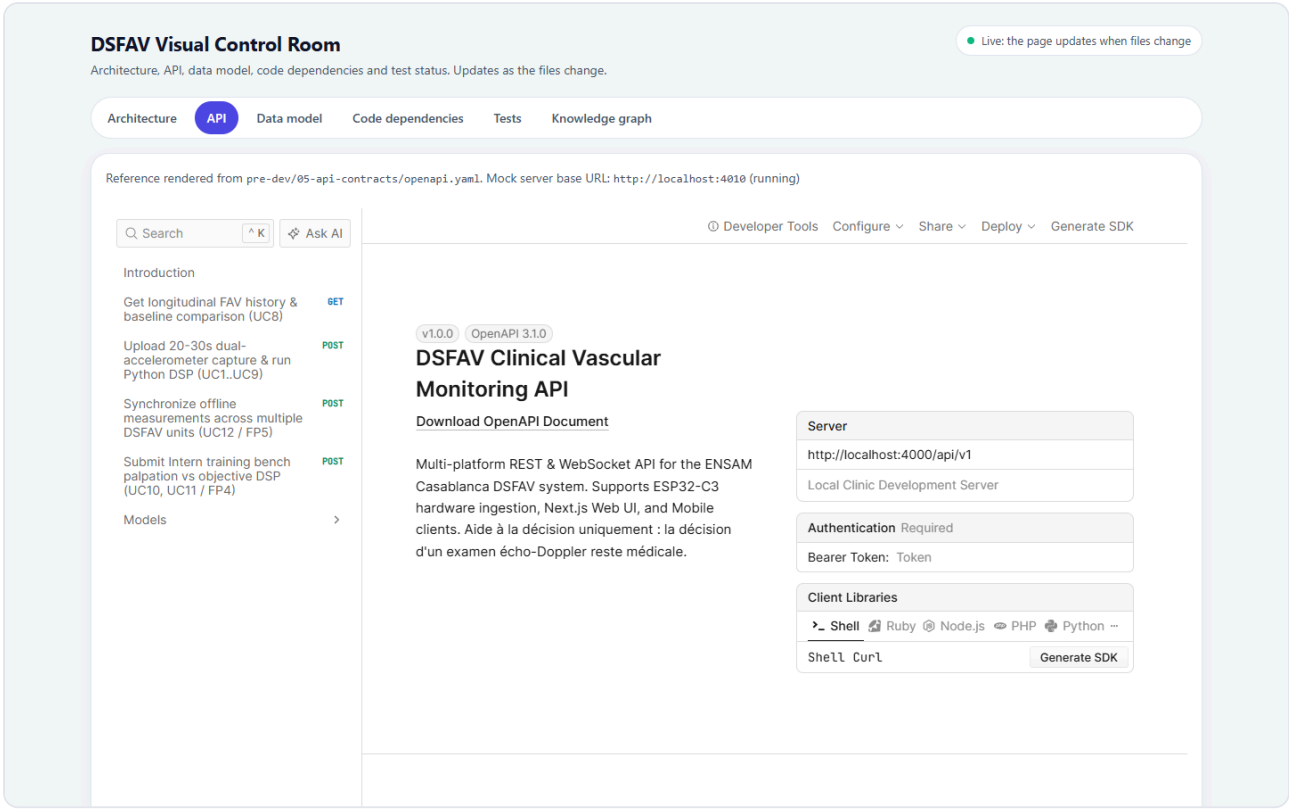
3. Démarche et méthode

Chaque vue est produite par un outil qui lit une source unique :

ONGLET	SOURCE	OUTIL
Architecture	`architecture/*.c4`	LikeC4 (serveur vivant, port 4610)
API	`pre-dev/05-api-contracts/openapi.yaml`	Scalar (rendu), Prism (serveur simulé, port 4010)
Modèle de données	`apps/api-nestjs/prisma/schema.prisma`	générateur Mermaid `gen/erd.mjs`
Dépendances	imports du code	dependency-cruiser, avec règles (pas de cycle, pas d'import web ↔ API)
Tests	`reports/junit/*.xml`	`gen/traceability.mjs` (LT-05)
Graphe	`graphify-out/graph.html`	graphify (LT-03)

Le serveur (port 4600) surveille les dossiers sources et relance le générateur concerné, puis notifie la page.

4. Conception



DSFAV Visual Control Room

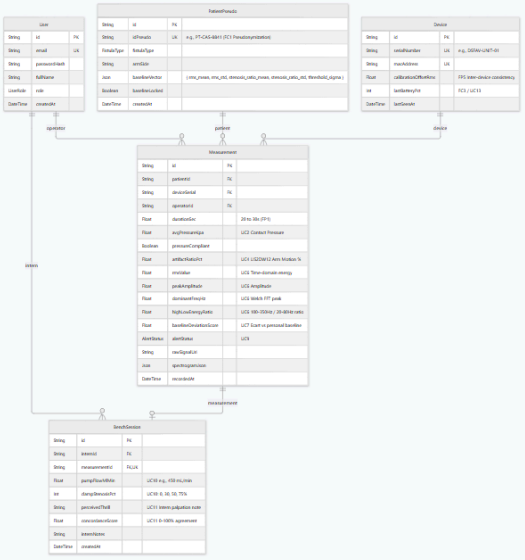
Live: the page updates when files change

Architecture, API, data model, code dependencies and test status. Updates as the files change.

Architecture | API | **Data model** | Code dependencies | Tests | Knowledge graph

Tables, fields and relations from the Prisma schema. Drag to pan, scroll to zoom.

\- | + | Fit | Source (.mmd)



DSFAV Visual Control Room

Live: the page updates when files change

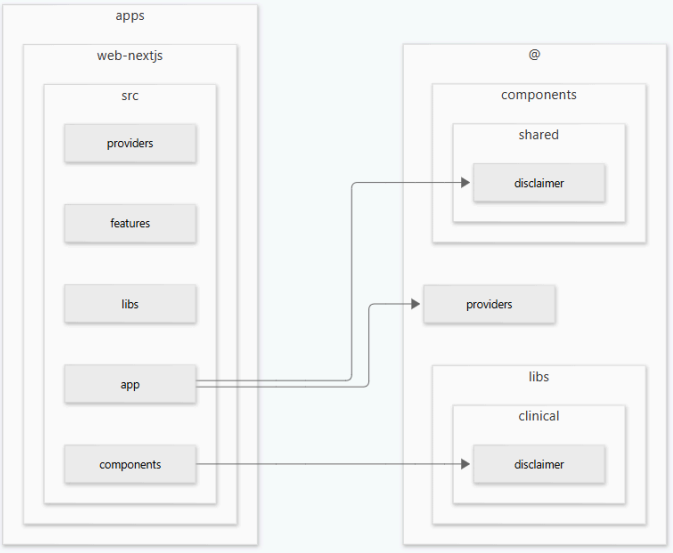
Architecture, API, data model, code dependencies and test status. Updates as the files change.

Architecture | API | Data model | **Code dependencies** | Tests | Knowledge graph

Scope: Web app (apps/web-nextjs/src)

Arrows point from the importing file to the file it uses. Rules live in tools/visual/.dependency-cruiser.cjs.

\- | + | Fit



5. Réalisation

`tools/visual/` (serveur, générateurs, page), `architecture/` (modèle C4 : 5 vues, du contexte au flux de capture UC1 à UC9), `.vscode/` (12 extensions recommandées, tâches « Visual: control room » et « Tests: all »), guide `docs/visual/README.md`.

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Validation et build du modèle C4	PASS	rapport de l'agent, `likec4 validate` / `build`
Modèle de données généré	5 modèles, 3 énumérations, 5 relations	sortie de `gen/erd.mjs`
Six onglets dans Edge	0 erreur de console	script Playwright, 2026-11-07
Onglet Architecture	défaut trouvé puis corrigé (voir §7)	captures avant/après

7. Décisions et justifications

Défaut corrigé : l'onglet Architecture annonçait « serveur non démarré » alors qu'il répondait. Cause : LikeC4 écoute sur l'adresse IPv6 `::1`, et la sonde interrogeait `127.0.0.1` (IPv4). Correction : sonder `localhost` (`tools/visual/server.mjs:85`). Ce piège figure dans les leçons du kit. Bibliothèques chargées depuis un CDN épinglées à des versions précises de plus de deux semaines (Mermaid 12.2.0, Scalar 1.76.0) pour la reproductibilité.

8. Limites et points ouverts

Le serveur simulé Prism ne recharge pas le contrat à chaud (redémarrer `pnpm visual:mock`) et met environ une minute à démarrer sur ce poste. Les liens du modèle C4 vers les documents fonctionnent dans l'aperçu vivant, pas dans la version statique.

9. Ce qu'il faut retenir

- Chaque diagramme est calculé depuis le code ou la spécification : il ne peut pas mentir longtemps.
- Une seule commande ouvre toute la vue d'ensemble.

10. Questions possibles du jury

- **Qu'est-ce que le modèle C4 ?** Quatre niveaux de zoom : qui utilise le système, quels conteneurs, quels composants, quel code ; nous utilisons les trois premiers.
- **À quoi sert le serveur simulé ?** L'interface peut être développée et démontrée contre l'API avant que le backend existe (« contrat d'abord »).

Références

`docs/visual/README.md` · `tools/visual/` · `architecture/`

LOT DE TRAVAIL 7

Système de design et génération des interfaces

Jalon M0 / M5 | en cours | 2026-11-07

Résumé

Les 8 écrans de DSFAV sont décrits par un système de design et des prompts de génération qui couvrent **8 écrans, 56 états et 4 visuels**, soit 68 prompts (`artifacts/dsfav-ui-previews/manifest.json`). Une première génération (v4) était fidèle au contenu mais trop dense ; à partir de deux références choisies par le porteur du projet, le langage visuel a été refondu (« Soft Bento », v5) sans modifier aucune règle clinique (D-006).

1. Contexte et problématique

L'écran d'acquisition est utilisé debout au fauteuil de dialyse, sur tablette. L'information doit être lisible en quelques secondes et la preuve (signal, qualité) doit précéder l'interprétation (`PRODUCT.md`, principes de design).

2. Objectifs

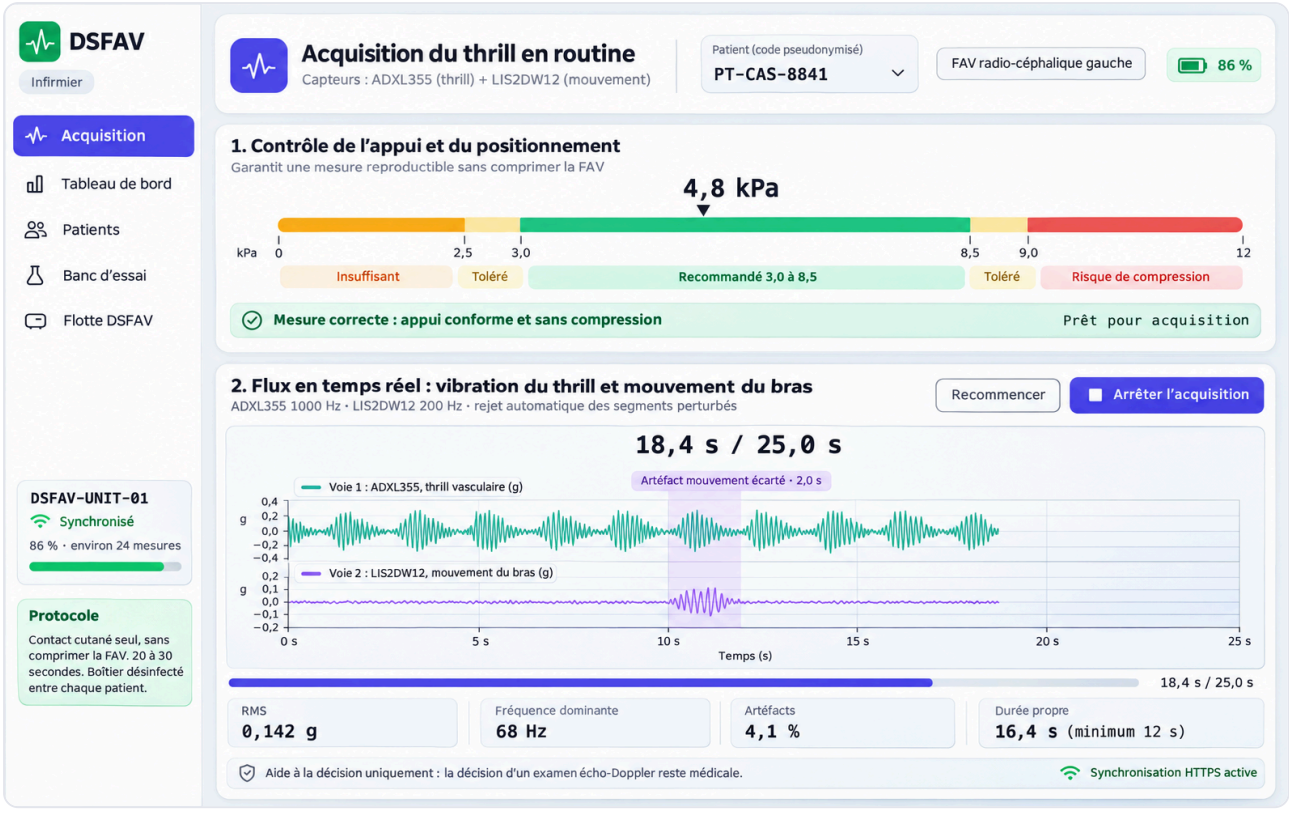
Des maquettes cohérentes entre écrans, conformes aux règles cliniques (zones de pression, mention légale, statuts), et proches du style visuel attendu.

3. Démarche et méthode

1. Système de design (couleurs, typographie, grille, composants, mouvements, accessibilité) puis données fictives partagées (`artifacts/DSFAV_FIXTURE.md`) pour que tous les écrans affichent les mêmes chiffres.
2. Un bloc commun + un prompt par écran + un prompt par état (pression basse, compression, rejet, hors ligne, baseline en construction…).
3. Génération, comparaison avec les références, correction du système puis des prompts.
4. Générateur de galerie (`tools/previews/generateDsfavPreviews.ts`) : en mode par défaut, il n'appelle aucun service payant ; la génération réelle exige l'option `--run` et une clé lue dans l'environnement.

4. Conception

Avant (v4, sortie réelle) :



Références cibles : `artifacts/design-references/01-target-medicare-teal-bento.png` et `02-target-medkit-soft-pills.png` .

5. Réalisation

- `artifacts/DSFAV_DESIGN.md` v4 « Soft Bento » : fond gris doux, cartes blanches sans bordure (rayon 28 px), grands chiffres légers avec unité grise, pastilles d'icônes, histogrammes en pilules avec barres de référence hachurées, navigation latérale avec pilule sarcelle, une seule carte sombre par écran, avant-bras 3D avec annotations.
- `artifacts/DSFAV_GENERATION_PROMPTS_V5.md` : prompts réécrits, mêmes contenus et états.
- Galerie : `artifacts/dsfav-ui-previews/index.html` (68 cartes, aucune requête réseau).

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Prompts analysés	8 écrans, 56 états, 4 visuels	sortie du générateur, 2026-11-07
Secrets dans les fichiers du générateur	aucun	recherche du 2026-11-07
Comparaison v4 / références	v4 : bordures, chiffres à chasse fixe, oscilloscope quadrillé, rangées de 4 valeurs	analyse des captures
Génération v5	non mesurée (à générer)	n/a

7. Décisions et justifications

D-006 : le style cible choisi par le porteur du projet prévaut sur l'ancienne liste d'« anti-références » de `PRODUCT.md`, qui a été mise à jour. Restent interdits : photos de visages, noms de patients, cartes commerciales, scores non justifiés. Le contenu clinique (chiffres, états, mention légale) est inchangé.

8. Limites et points ouverts

Les maquettes v5 restent à générer et à comparer aux références. Les maquettes ne sont pas l'application : les écrans réels seront construits au jalon M5 et testés (accessibilité WCAG 2.2 AA, lecture à 200 %).

9. Ce qu'il faut retenir

- Un système de design est un ensemble de règles, pas une image.
- On change le style sans toucher aux règles cliniques.

- La preuve (signal, qualité) apparaît avant l'interprétation.

10. Questions possibles du jury

- **Pourquoi des états séparés ?** Les cas d'erreur (appui insuffisant, compression, mouvement, hors ligne) sont ceux où l'interface protège le patient ; ils doivent être conçus, pas improvisés.
- **Comment éviter qu'une couleur seule porte une information ?** Chaque statut combine icône, mot et teinte (WCAG).

Références

`PRODUCT.md` · `artifacts/DSFAV_DESIGN.md` · `artifacts/DSFAV_GENERATION_PROMPTS_V5.md` · `docs/DECISIONS.md` D-006

LOT DE TRAVAIL 8

Pack de contexte et documentation académique

Jalon M0 | terminé | 2026-11-07

Résumé

Deux mécanismes de documentation ont été mis en place. Le **pack de contexte** regroupe tout le projet en 17 fichiers (environ 1,1 Mo) pour l'importer dans un assistant externe. Le **dossier académique** (ce dossier) produit un rapport par lot de travail, rendu en PDF au format A4. Les deux se régénèrent par une commande (`pnpm context-pack` , `pnpm academic`).

1. Contexte et problématique

Le projet doit pouvoir être expliqué à un enseignant, repris par un autre membre de l'équipe, et compris par un assistant d'IA externe, sans relire la conversation de travail.

2. Objectifs

- Un ensemble de fichiers autosuffisant et à jour, avec un ordre d'autorité explicite.

- Un rapport académique par étape, dont chaque chiffre cite sa source.

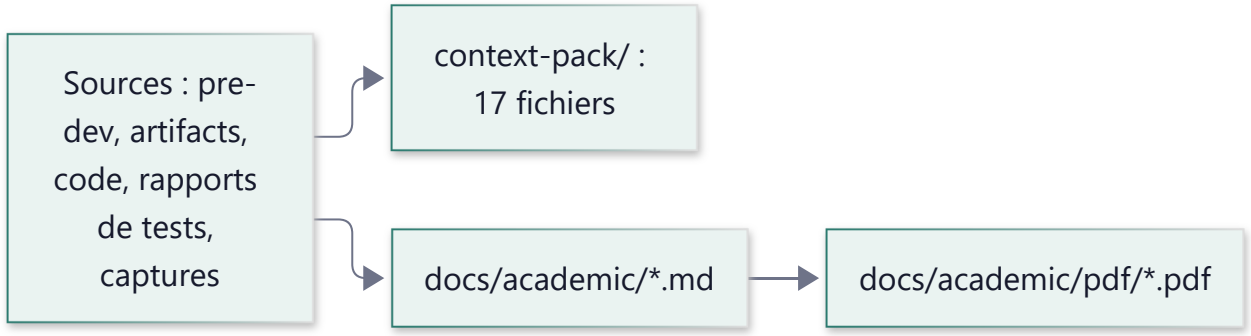
---

3. Démarche et méthode

- **Pack de contexte** (`tools/context-pack/build.mjs`) : concatène les documents par thème avec, avant chaque extrait, le chemin du fichier source ; ajoute l'arborescence du dépôt, les configurations, un instantané du code et des instructions prêtes à coller. Les assistants limitent le nombre de fichiers, d'où le regroupement.
- **Dossier académique** : combine la logique « tuteur » du kit (expliquer simplement, citer le code réel, vérifier la compréhension) et la règle des rapports de session du kit (chaque chiffre cite sa source, aucun résultat inventé) ; structure imposée par `.claude/skills/dsfav-academic/SKILL.md` .

---

4. Conception



---

5. Réalisation

- `context-pack/00-README-START-HERE.md` (présentation, ordre d'autorité, instructions) à `16-source-code-snapshot.md` , plus le cahier des charges en PDF.
- `docs/academic/` : sommaire, glossaire et lots LT-01 à LT-08 ; rendu PDF par `tools/academic/build.mjs` (navigateur Edge sans interface, diagrammes Mermaid rendus).
- Le tuteur du projet (`.agent-hub/tutor/TUTOR.md`) dispose d'un mode « soutenance » qui interroge l'étudiant à partir des questions de chaque lot.

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Pack de contexte	17 fichiers, environ 1 100 Ko	sortie de `build.mjs`
Secrets dans le pack	aucun ; le document source brut, qui contient un identifiant Wi-Fi, est exclu du pack et ignoré par git	recherche du 2026-11-07, `.gitignore`

7. Décisions et justifications

Le dossier est rédigé en français, comme le cahier des charges et l'interface (`.agent-hub/tutor/TUTOR.md` §9).

8. Limites et points ouverts

Le pack et le dossier sont des instantanés : ils doivent être régénérés après chaque jalon. Les rapports rédigés par une IA doivent être relus par l'équipe avant présentation.

9. Ce qu'il faut retenir

- Documenter, c'est rendre le projet transmissible.
- Un chiffre sans source n'entre pas dans le dossier.

10. Questions possibles du jury

- **Comment savez-vous que ces rapports sont exacts ?** Chaque valeur renvoie à un fichier ou à une commande que l'on peut relancer.
- **Ces documents seront-ils tenus à jour ?** Un rapport par jalon, produit à la clôture du jalon, régénération par une commande.

Références

`tools/context-pack/build.mjs` · `.claude/skills/dsfav-academic/SKILL.md` · `.agent-hub/tutor/TUTOR.md`

PARTIE 2

Réalisation du système

Contrat d'API, traitement du signal, cœur du backend, application web, banc de formation et démonstration.

LOT DE TRAVAIL 9

Contrat d'API et modèle de données (jalon M1)

Jalon M1 | terminé | 2026-11-07

Résumé

Le jalon M1 « gel du contrat » a fixé, avant tout code de fonctionnalité, les trois interfaces dont dépendent toutes les équipes : l'API publique (**20 chemins, 22 opérations**, OpenAPI 3.1 valide), la base de données (**11 modèles, 8 énumérations**, une migration appliquée et un jeu de données de démonstration) et le protocole de communication de l'appareil (document de 198 lignes). Le contrat a été validé par l'outil Redocly et un test automatique vérifie qu'aucun champ nominatif n'existe (sources en §6).

1. Contexte et problématique

Le plan impose « le contrat d'abord » (`pre-dev/10-build-plan.md` §1) : plusieurs agents construisent en parallèle l'API, l'interface et le simulateur. Sans interface figée, chacun devine les formes de données des autres et l'intégration échoue.

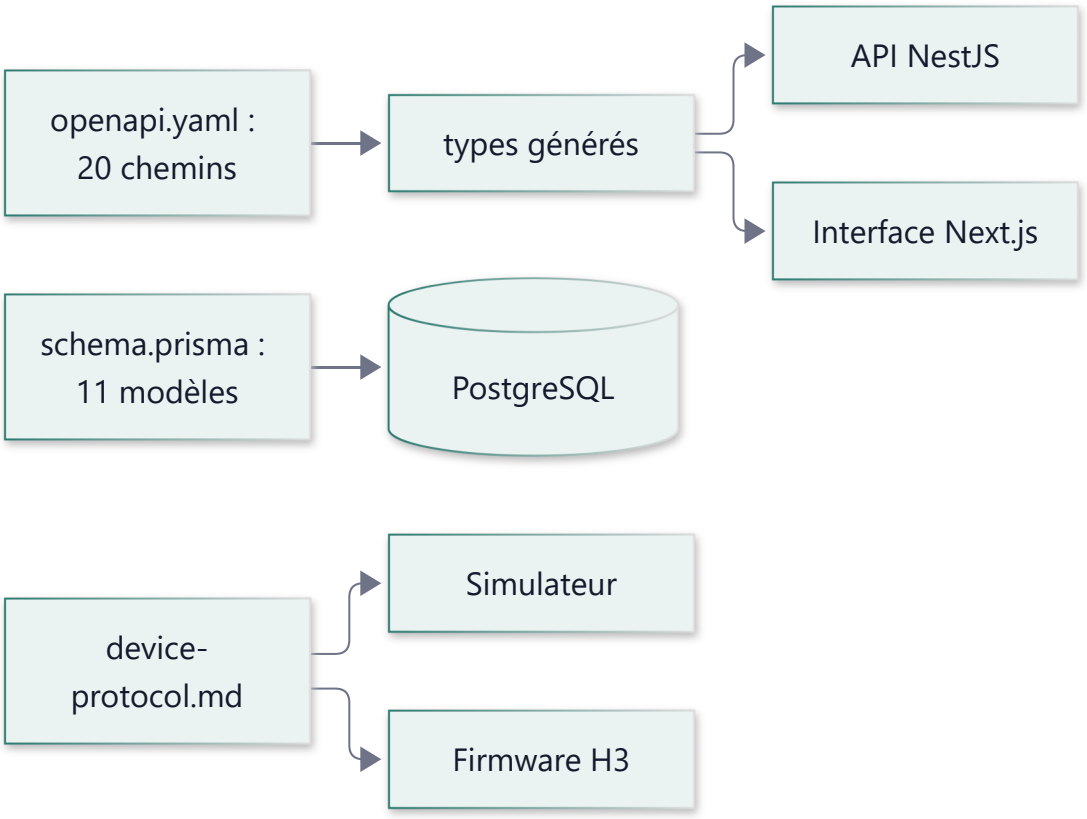
2. Objectifs

- Un contrat d'API unique, valide et suffisant pour les écrans (critère : validation sans erreur).
- Un schéma de base conforme à la spécification canonique §6 et **sans donnée personnelle** (règle FC1).
- Un protocole appareil ↔ serveur lisible par l'équipe matérielle (tâche H3).

3. Démarche et méthode

La tâche T-s01 a été confiée à un agent exécutant, puis relue par un agent indépendant (3 tours). Le contrat OpenAPI est la source ; les types TypeScript de l'interface sont **générés** depuis lui (`packages/shared-types`), jamais écrits à la main.

4. Conception



Modèles : User, PatientPseudo, Device, BaselineVersion, ConfigVersion, Measurement, ClinicalNote, FollowUpIntent, AuditEvent, SyncQueueItem, BenchSession. Énumérations : UserRole, FistulaType, AlertStatus, RejectReason, PerceivedThrill, MeasurementSource, SyncState, ConfigStatus.

5. Réalisation

- `pre-dev/05-api-contracts/openapi.yaml` : routes de la spécification canonique §7 (authentification, mesures en deux étapes, patients, appareils, banc, configuration), corps d'erreur unique `{code, message_fr, retryable, field?}` , champ `disclaimer` sur toute réponse portant une mesure.

- `apps/api-nestjs/prisma/schema.prisma` et la migration `20261107141111_v1` ; `prisma/seed.ts` crée 4 utilisateurs (un par rôle), 2 patients pseudonymisés, 4 appareils, une configuration active et une baseline verrouillée.
- `docs/protocol/device-protocol.md` : trames WebSocket de l'appareil et format binaire de l'envoi brut (en-tête, échantillons entiers 16 bits, pression en centi-kPa).
- `apps/api-nestjs/test/pseudo-scan.spec.ts` : lit le schéma Prisma et le contrat et **échoue** si un champ nominatif apparaît (nom, prénom, CIN, date de naissance, téléphone, adresse) ; des tests négatifs prouvent qu'il détecte `name` et `firstName`.

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Validation du contrat	valide (0 erreur)	`pnpm contract` (Redocly), 2026-11-07
Chemins / opérations	20 / 22	comptage dans `openapi.yaml`
Modèles / énumérations	11 / 8	`schema.prisma`
Test PSEUDO-SCAN	5 tests passent	`pnpm --filter @dsfav/api test`
Données de démonstration	4 utilisateurs, 2 patients, 4 appareils, 1 configuration, 1 baseline	requête sur la base, 2026-11-07
Client Prisma réel	lecture réussie via l'adaptateur `pg`	script de vérification, 2026-11-07

7. Décisions et justifications

- **Prisma 7 exige un adaptateur de pilote** pour joindre PostgreSQL ; il n'était pas installé et un agent avait contourné le problème par une base simulée en mémoire. L'orchestrateur a installé `@prisma/adapter-pg` et `pg`, vérifié la lecture réelle, puis renvoyé la tâche en reprise (voir LT-02 pour le processus de revue).
- **Contrat avant code** : les écrans ont découvert deux manques que le gel n'avait pas prévus (aucune route de lecture des notes ; aucune donnée de synchronisation dans la synthèse d'appareil). Ils sont consignés dans `docs/BACKLOG.md` plutôt que contournés dans l'interface.

8. Limites et points ouverts

Le contrat n'est pas encore implémenté par l'API : l'historique d'un patient renvoie pour l'instant des lignes brutes de base, alors que le contrat définit une autre forme ; la correction est confiée à la tâche T-s07. Le seuil d'alerte et le temps de recharge restent des valeurs par défaut « à calibrer au banc ».

9. Ce qu'il faut retenir

- Une interface figée en premier permet de construire en parallèle sans se marcher dessus.
- Un test automatique garantit l'absence de données personnelles, à chaque exécution.
- Un contrat peut être valide et pourtant incomplet : ce sont les écrans qui l'ont montré.

10. Questions possibles du jury

- **Pourquoi générer les types plutôt que les écrire ?** Le compilateur signale alors tout écart entre l'interface et l'API.
- **Comment garantir l'absence de données nominatives ?** Le test PSEUDO-SCAN échoue à la moindre apparition d'un champ de ce type, dans le schéma comme dans le contrat.
- **Pourquoi des données de démonstration dans la base ?** Pour que la démonstration et les tests partent d'un état connu ; les mots de passe de développement ne figurent que dans le jeu de données et le guide de démarrage.

Références

`pre-dev/05-api-contracts/openapi.yaml` · `apps/api-nestjs/prisma/schema.prisma` · `docs/protocol/device-protocol.md` · `docs/DECISIONS.md` · `pre-dev/09-canonical-spec.md` §6-7

LOT DE TRAVAIL 10

Moteur de traitement du signal (jalon M2)

Jalon M2 | terminé | 2026-11-08

Résumé

Le jalon M2 a produit le moteur qui transforme une capture brute (accéléromètre à 1 kHz, accéléromètre de mouvement à 200 Hz, pression) en un enregistrement contrôlé : porte de pression, porte qualité, caractéristiques du signal et écart à la référence personnelle du patient. Le moteur est une bibliothèque Python pure (`dsfav_dsp`, environ 800 lignes) exposée par une petite API. Ses **25 tests passent** (9 tests « golden » G1 à G9, tests de propriétés, test de performance, tests de l'API), le traitement de 25 s de signal prend **0,08 s** pour un budget de 1,5 s (sources en §6).

1. Contexte et problématique

La décision d'une mesure repose sur des règles chiffrées : pression valide de 3,0 à 8,5 kPa, mouvement rejeté au-delà de 0,08 g RMS par fenêtre d'une seconde, rejet sous 12 s de signal propre. Ces règles doivent être vérifiables sans matériel : le dispositif n'existe pas encore, donc le moteur est testé sur des signaux synthétiques dont on connaît la bonne réponse.

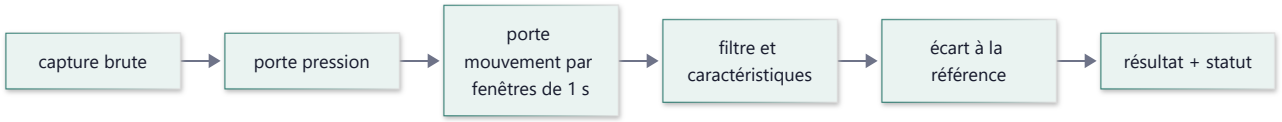
2. Objectifs

- Appliquer les règles de la spécification canonique à l'identique, avec des seuils lus dans la configuration et non écrits dans le code.
- Prouver le calcul (RMS, fréquence dominante, rapport hautes/basses fréquences) par des tests dont la réponse est connue d'avance.
- Rester sous 1,5 s par séance pour que l'API puisse l'appeler en synchrone.

3. Démarche et méthode

Les tests ont été écrits **avant** le code (rouge puis vert). Un générateur de signaux synthétiques (`synth.py`) crée des cas contrôlés : signal propre, rafale de mouvement, pression instable. Les tests G1 à G9 fixent le comportement attendu ; les tests de propriétés vérifient des invariants sur des entrées aléatoires.

4. Conception



Modules : `gate.py` (pression et qualité), `filters.py`, `features.py`, `baseline.py` (référence personnelle), `pipeline.py` (enchaînement), `api.py` (exposition).

5. Réalisation

- **Gravité retirée axe par axe** : l'accéléromètre de mouvement mesure aussi la pesanteur (1 g). La médiane de chaque axe est retranchée avant le calcul du RMS. Une première idée (médiane de la norme) échouait sur les longues rafales ; elle a été abandonnée après le test G4.
- **Fenêtres alignées de 1 s, sans recouvrement** (décision D-011) : une rafale de 2 s masque exactement 2 secondes, ce qui rend le test G3 déterministe.
- **Gain** : le gain de calibration reste un multiplicateur ; l'API envoie l'inverse du gain de l'appareil, d'où un traitement identique d'un appareil à l'autre.
- Les erreurs d'entrée renvoient un message en français (`INVALID_REQUEST`).

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Tests du moteur	25 passent, 0 échec	`uv run pytest` dans `apps/dsp-python`, 2026-11-08
Tests golden G1 à G9	9 définis, tous passent	`tests/golden/test_golden.py`
Performance	0,08 s pour 25 s de signal (budget 1,5 s)	`tests/test_performance.py`
Revue indépendante	PASS (Gemini), 2 remarques non bloquantes	journal du hub, tâche T-s04

7. Décisions et justifications

- **Tests avant code** : sans signal réel, seul un signal fabriqué dont on connaît le résultat prouve que le calcul est juste.

- **Seuils en configuration** : ils sont marqués « à calibrer au banc » dans la spécification et doivent pouvoir changer sans modifier le code.
- **Une erreur d'orchestration corrigée** : une consigne de ma part (diviser par le gain) contredisait les tests ; elle a été retirée et le test fait foi.

---

8. Limites et points ouverts

Les tests G1 à G9 **prouvent le calcul, pas la médecine** : les signaux sont synthétiques. Aucune valeur clinique ne peut être déduite avant l'étude sur banc (jalon M9). Remarque ouverte de la revue : la durée en échantillons de pression utilise une division entière ; un arrondi serait plus fidèle sur les limites d'une seconde.

---

9. Ce qu'il faut retenir

- Un moteur de calcul se prouve avec des signaux dont on connaît la réponse.
- Les seuils cliniques sont de la configuration, pas du code.
- Un test qui contredit la consigne du chef de projet doit être écouté, pas contourné.

---

10. Questions possibles du jury

- **Pourquoi des signaux synthétiques ?** Le dispositif n'existe pas encore ; le synthétique fournit une réponse connue pour vérifier le calcul.
- **Cela prouve-t-il que l'outil détecte une sténose ?** Non. Cela prouve que le calcul est exact ; la validité clinique relève de l'étude sur banc.
- **Pourquoi retirer la gravité axe par axe ?** La pesanteur s'ajoute à chaque axe ; la médiane de chaque axe l'estime sans être gênée par une rafale courte.

Références

`apps/dsp-python/src/dsfav_dsp/` · `apps/dsp-python/tests/` · `docs/DECISIONS.md` (D-011 à ajouter) · `pre-dev/09-canonical-spec.md`

LOT DE TRAVAIL 11

Cœur backend et pipeline de mesure

Jalon M3 | en cours | 2026-11-08

Résumé

Le backend relie l'authentification, les patients pseudonymisés, les mesures, la référence personnelle et le service DSP. Les modules NestJS sont enregistrés dans l'application et les tests couvrent les rôles, la sérialisation, la référence et la persistance. La validation d'intégration reste partielle : plusieurs preuves API n'ont pas démarré à cause de la résolution locale de `@prisma/adapter-pg` (journal du hub, T-s05, T-s07, T-s11, T-s12, T-s19).

1. Contexte et problématique

Le système doit conserver chaque tentative de mesure et fournir une réponse structurée, avec un statut et les éléments nécessaires à l'interprétation médicale. Le plan M3 relie l'authentification par rôles, les patients, le cycle de référence et le traitement DSP (pré-dev/10-build-plan.md, §3 M3).

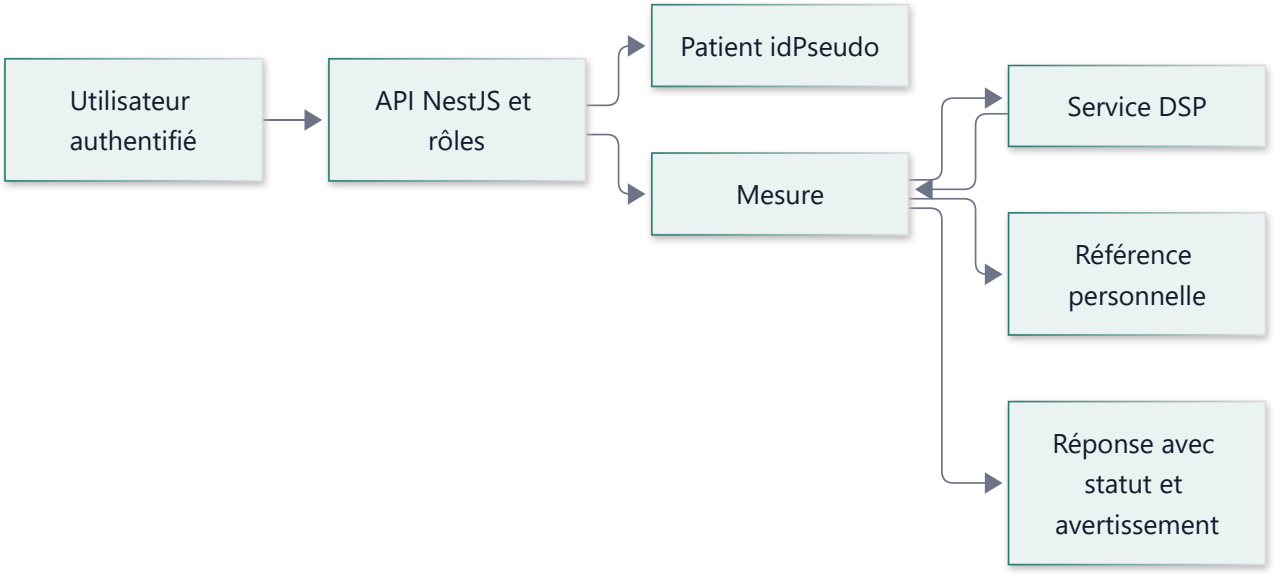
2. Objectifs

- Enregistrer les mesures et leurs statuts, puis les comparer à la référence propre au patient (pré-dev/09-canonical-spec.md, §6; `apps/api-nestjs/src/modules/measurements/`).
- Contrôler l'accès selon la matrice des rôles, couverte par `[AUTHZ-MATRIX]` (journal du hub, T-s05; `apps/api-nestjs/test/authz-matrix.e2e-spec.ts`).
- Conserver l'avertissement clinique dans les réponses via la constante partagée du backend (`apps/api-nestjs/src/common/constants.ts:2-3`).

3. Démarche et méthode

Les travaux acceptés ont relié les modules réels et renforcé les tests ciblés. Le rapport distingue les tests exécutés et ceux empêchés avant leur démarrage, selon les preuves consignées par tâche (journal du hub, événements `done` de T-s05, T-s07, T-s19).

4. Conception



Le service de mesure appelle le client DSP, puis sérialise le résultat. Les routes de référence et d'audit restent dans des modules séparés (`apps/api-nestjs/src/modules/measurements/measurements.service.ts` ; `apps/api-nestjs/src/modules/measurements/dsp.client.ts` ; `apps/api-nestjs/src/modules/measurements/measurement.serializer.ts`).

5. Réalisation

Les tâches acceptées ont relié Prisma à la base existante, les modules métier et les protections d'accès (journal du hub, T-s05 et T-s19). Le pipeline de signal est une bibliothèque Python indépendante et un service FastAPI (`apps/dsp-python/src/dsfav_dsp/pipeline.py` ; `apps/dsp-python/src/dsfav_dsp/api.py`). La référence est construite et verrouillée par la logique backend (`apps/api-nestjs/src/modules/baseline/baseline.logic.ts`).

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Matrice d'autorisations et tests API ciblés	64 tests passent selon le compte rendu; la suite API du builder a réussi	journal du hub, T-s05, événement `done`
Pipeline DSP	25 tests passent; test de performance mesuré sous la limite NFR-01 dans ce run	journal du hub, T-s04; `apps/dsp-python/tests/test_performance.py` ; pré-dev/10-build-plan.md, §3 M2
Intégration API indépendante	Non mesurée dans certains runs : Jest s'arrête avant les tests sur `@prisma/adapter-pg`	journal du hub, T-s07, T-s11, T-s12, T-s19
Lint et types API	Typecheck sans erreur ; suite complète 117 tests passés, 0 échec, 8 ignorés (trois exécutions consécutives)	`pnpm --filter @dsfav/api test` et `tsc --noEmit` , 2026-11-08

Les résultats de tests prouvent le comportement logiciel dans les conditions du test, pas une performance ou une efficacité clinique.

7. Décisions et justifications

Le traitement DSP est synchrone dans le déploiement de démonstration; Redis, MinIO et les fonctionnalités Timescale ne sont pas utilisés dans la première version (docs/DECISIONS.md, D-007). Les seuils restent configurables et doivent être calibrés au banc (AGENTS.md, §4 règle 6; pré-dev/09-canonical-spec.md).

8. Limites et points ouverts

La résolution de `@prisma/adapter-pg` a empêché plusieurs suites d'intégration de démarrer; ces cas ne sont donc pas déclarés réussis (journal du hub, T-s07, T-s11, T-s12, T-s19). La cohérence clinique des seuils n'est pas démontrée par les fixtures synthétiques (pré-dev/10-build-plan.md, §1 principes 4 et 6).

9. Ce qu'il faut retenir

- Le backend enregistre des mesures sous identifiant pseudonyme.
- Le calcul de signal et la persistance sont des responsabilités distinctes.
- Un test qui n'a pas démarré ne constitue pas une preuve de réussite.
- Les essais synthétiques prouvent une partie du calcul, pas une décision médicale.

10. Questions possibles du jury

- **Pourquoi isoler le DSP ?** Pour tester le calcul sans dépendre de l'interface (`apps/dsp-python/src/dsfav_dsp/`).
- **La matrice des rôles est-elle vérifiée ?** Oui : la matrice AUTHZ-MATRIX est exécutée par la suite e2e de l'API (117 tests passés au total le 2026-11-08, `pnpm --filter @dsfav/api test`). Les blocages signalés pendant la construction venaient du bac à sable des agents, qui ne suit pas les liens pnpm, et non du code.
- **Les seuils sont-ils cliniquement validés ?** Non; ils doivent être calibrés au banc (AGENTS.md, §4 règle 6).

Références

`apps/api-nestjs/src/modules/` · `apps/dsp-python/src/dsfav_dsp/` · `apps/api-nestjs/test/authz-matrix.e2e-spec.ts` · `.agent-hub/events.jsonl` (T-s04, T-s05, T-s07, T-s11, T-s12, T-s19) · `docs/DECISIONS.md` · `pre-dev/09-canonical-spec.md` · `pre-dev/10-build-plan.md`

LOT DE TRAVAIL 12

Application web et système de design

Jalon M5 | terminé | 2026-11-08

Résumé

L'application Next.js fournit les parcours de connexion, capture, revue patient, file de synchronisation et administration des appareils. Les vues utilisent des composants réutilisables et des contenus en français. Les dernières preuves enregistrées couvrent 16 fichiers de test et 98 tests web; des vérifications plus anciennes signalent des échecs transitoires dans les travaux d'autres voies (journal du hub, T-s17, T-s18, T-s20).

1. Contexte et problématique

L'infirmier capture la mesure sur tablette; le néphrologue consulte l'historique; l'interne utilise le banc de formation (AGENTS.md, §3). L'interface doit expliquer les états de qualité et ne pas afficher de valeurs objectives fictives en mode réel (`apps/web-nextjs/src/features/capture/`).

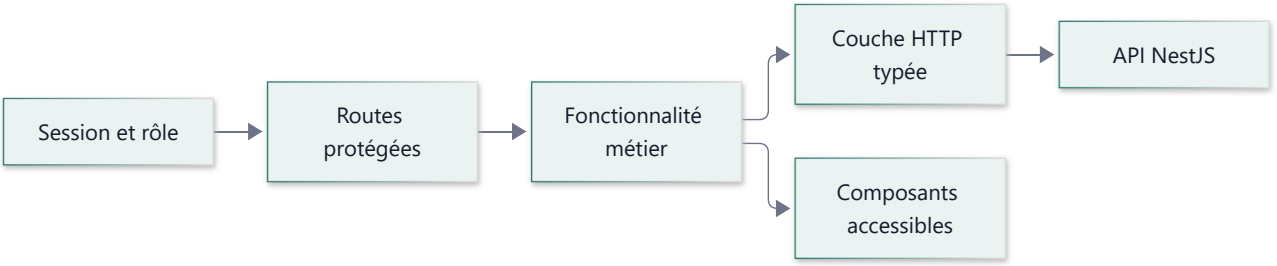
2. Objectifs

- Présenter les parcours principaux selon les rôles et les règles d'accès (pré-dev/09-canonical-spec.md, §10; `apps/web-nextjs/src/proxy.ts`).
- Afficher des données typées et des états d'attente ou d'indisponibilité (`apps/web-nextjs/src/features/`; journal du hub, T-s13 et T-s20).
- Garder les libellés cliniques en français et afficher l'avertissement partagé (`apps/web-nextjs/src/libs/clinical/disclaimer.ts`; AGENTS.md, §5 règle 8).

3. Démarche et méthode

Les changements ont été développés par fonctionnalité avec tests Vitest ciblés et vérification de types/lint. Le système de composants conserve l'accès clavier et les noms accessibles dans les contrôles concernés (journal du hub, T-s18; `apps/web-nextjs/src/components/`).

4. Conception



Les pages serveur chargent les données et délèguent les interactions locales aux composants client (`apps/web-nextjs/src/app/(clinic)/`; `apps/web-nextjs/src/features/`).

5. Réalisation

Les vues incluent notamment capture, patients, appareils, banc et historique de référence (`apps/web-nextjs/src/features/`). Les routes de chargement différé utilisent des frontières Suspense pour plusieurs écrans de clinique (`apps/web-nextjs/src/app/(clinic)/dashboard/page.tsx`; T-s20). Le proxy de session est dans `src/proxy.ts` après migration du middleware (journal du hub, T-s16).

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Suite web rapportée après les tâches UI	16 fichiers, 98 tests passés	journal du hub, T-s18 et T-s20, événements `done`
Types web	`tsc --noEmit` terminé sans erreur sur les tâches concernées	journal du hub, T-s17, T-s18, T-s20
Lint web	Zéro erreur; deux avertissements préexistants dans un run	journal du hub, T-s17 et T-s20
Chargement de données cliniques	La page patient dépend d'un GET de notes absent du contrat gelé; point signalé au backlog	`docs/BACKLOG.md` ; journal du hub, T-s10

7. Décisions et justifications

Les bibliothèques d'interface et de graphiques retenues sont celles approuvées par D-010 (`docs/DECISIONS.md` , D-010). Le tableau de bord conserve une présentation de l'état de synchronisation sans inventer de mécanisme de reprise que le contrat ne fournit pas (`docs/BACKLOG.md` , entrée sync-queue; T-s13).

8. Limites et points ouverts

Les données manquantes du contrat peuvent limiter la lecture des notes et la reprise de synchronisation (`docs/BACKLOG.md` ; journal du hub, T-s10 et T-s13). Les données d'exemple visibles en mode démo sont identifiées comme telles; les mesures synthétiques ne démontrent pas une utilité clinique (`apps/web-nextjs/src/features/capture/data/demo-capture.ts` ; AGENTS.md, §4 règle 10).

9. Ce qu'il faut retenir

- Les routes et les données restent liées au rôle connecté.
- Les états d'attente et d'indisponibilité sont présentés à l'écran.
- Les exemples synthétiques servent à la démonstration seulement.
- Les lacunes de contrat restent des points ouverts visibles.

10. Questions possibles du jury

- **Pourquoi séparer pages et fonctionnalités ?** Pour garder le chargement des données dans les routes et les interactions dans les fonctionnalités (`apps/web-nextjs/src/app/` ; `src/features/`).
- **Que se passe-t-il si une donnée n'existe pas dans l'API ?** L'interface affiche l'indisponibilité plutôt qu'une donnée inventée (journal du hub, T-s13).
- **La démonstration prouve-t-elle une performance clinique ?** Non; elle prouve seulement les parcours logiciels sur données synthétiques (AGENTS.md, §4 règle 10).

Références

`apps/web-nextjs/src/app/` · `apps/web-nextjs/src/features/` · `apps/web-nextjs/src/components/` · `docs/DECISIONS.md` · `docs/BACKLOG.md` · `.agent-hub/events.jsonl` (T-s10, T-s13, T-s16, T-s17, T-s18, T-s20)

LOT DE TRAVAIL 13

Banc de formation et concordance

Jalon M7 | en cours | 2026-11-08

Résumé

Le banc propose un parcours d'entraînement aveugle : l'interne enregistre sa palpation avant que les résultats objectifs soient révélés. Une fonction calcule une concordance pondérée entre sensation, intensité et notes. Les profils et références d'intensité sont explicitement provisoires et à calibrer; aucune interprétation clinique n'en découle (`apps/api-nestjs/src/modules/bench/bench.constants.ts`).

1. Contexte et problématique

L'entraînement vise à comparer une perception de thrill à une mesure simulée. Le serveur doit empêcher l'accès aux données objectives avant la réponse de palpation (AGENTS.md, §4 règle 9; `apps/api-nestjs/src/modules/bench/bench.service.ts`).

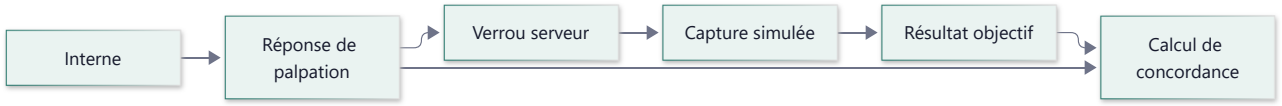
2. Objectifs

- Enregistrer la perception et appliquer la règle de révélation côté serveur (AGENTS.md, §4 règle 9).
- Calculer un score de concordance à partir de catégories, d'intensité et de notes (`apps/api-nestjs/src/modules/bench/bench.concordance.ts`).
- Marquer comme provisoires les paramètres de banc en attendant une calibration réelle (`apps/api-nestjs/src/modules/bench/bench.constants.ts`).

3. Démarche et méthode

Les tests de logique examinent le calcul de concordance; les tests d'intégration vérifient les transitions de banc et les refus d'accès (`apps/api-nestjs/src/modules/bench/bench.concordance.spec.ts` ; `apps/api-nestjs/src/modules/bench/bench.e2e-spec.ts`). L'API utilise une mise à jour conditionnelle pour éviter d'écraser une palpation ou une capture déjà prise (journal du hub, T-s12).

4. Conception



Le score se décompose en sensation, intensité et notes (`apps/api-nestjs/src/modules/bench/bench.concordance.ts`).

5. Réalisation

L'échelle de sensation et les profils attendus sont déclarés dans les constantes du module. Les pondérations sont 60 %, 25 % et 15 % respectivement, des valeurs de démonstration présentes dans le code et non des paramètres validés cliniquement (`apps/api-nestjs/src/modules/bench/bench.constants.ts`). La capture du banc est désactivée si l'identifiant d'appareil configuré est absent ou invalide (journal du hub, T-s17).

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Tests unitaires de concordance	Sept tests ciblés passent dans un run	journal du hub, T-s12
Tests de protection concurrente	Tests de concurrence passés dans la suite complète de l'API (117 passés le 2026-11-08) ; l'avertissement initial venait du bac à sable de l'agent	`pnpm --filter @dsfav/api test`
Interface du banc	Suite web rapportée à 98 tests passés lors de runs finaux	journal du hub, T-s17 et T-s18

7. Décisions et justifications

Les profils attendus et seuils d'intensité sont explicitement nommés `PLACEHOLDER` ; les remplacer nécessite des mesures de banc et une décision documentée (code `bench.constants.ts` ; AGENTS.md, §4 règle 6). Le résultat est une aide pédagogique et ne correspond pas à une conclusion médicale (AGENTS.md, §4 règle 10).

8. Limites et points ouverts

Les profils et références restent des valeurs de remplacement à calibrer. La concordance vérifie la cohérence avec les attentes programmées, pas la validité des attentes ni la compétence clinique (code `bench.constants.ts` ; `.ultraplan/sections/section-09-bench-study-kit.md`). L'intégration serveur concurrente n'a pas été entièrement prouvée dans le shell du builder en raison de la résolution Prisma (journal du hub, T-s12).

9. Ce qu'il faut retenir

- La réponse de palpation précède la révélation objective.
- Le score décrit un écart aux profils configurés.
- Les profils configurés sont provisoires.
- Le banc ne produit aucune preuve médicale.

10. Questions possibles du jury

- **Pourquoi cacher le résultat objectif au départ ?** Pour préserver un exercice de palpation indépendant (AGENTS.md, §4 règle 9).
- **Que signifie le score ?** Une concordance calculée entre trois composantes pondérées (`bench.concordance.ts` ; `bench.constants.ts`).
- **Les profils sont-ils validés ?** Non, ils sont explicitement provisoires et doivent être calibrés au banc (`bench.constants.ts`).

Références

`apps/api-nestjs/src/modules/bench/` · `apps/web-nextjs/src/features/bench/` · `.agent-hub/events.jsonl` (T-s12, T-s17, T-s18) · `.ultraplan/sections/section-09-bench-study-kit.md` · `AGENTS.md` , §4

LOT DE TRAVAIL 14

Intégration et démonstration de bout en bout

Jalon M5 | terminé | 2026-11-08

Résumé

La suite Playwright décrit huit étapes de démonstration et un scénario de banc. Le compte rendu d'intégration fait état de dix tests e2e réussis sur la pile active (journal du hub, T-s14). L'étape de coupure réseau vérifie l'arrêt avant téléversement brut; elle ne prouve pas la mise en file et la reprise attendues d'un scénario de perte de session (`e2e/tests/demo.spec.ts` ; `docs/BACKLOG.md`).

1. Contexte et problématique

Une démonstration reproductible relie simulateur, API, DSP et interface. Le plan prévoit les scénarios de capture normale, pression, mouvement, évolution, historique, banc et perte de connexion (`pre-dev/10-build-plan.md` , §8; `.ultraplan/sections/section-14-demo-e2e.md`).

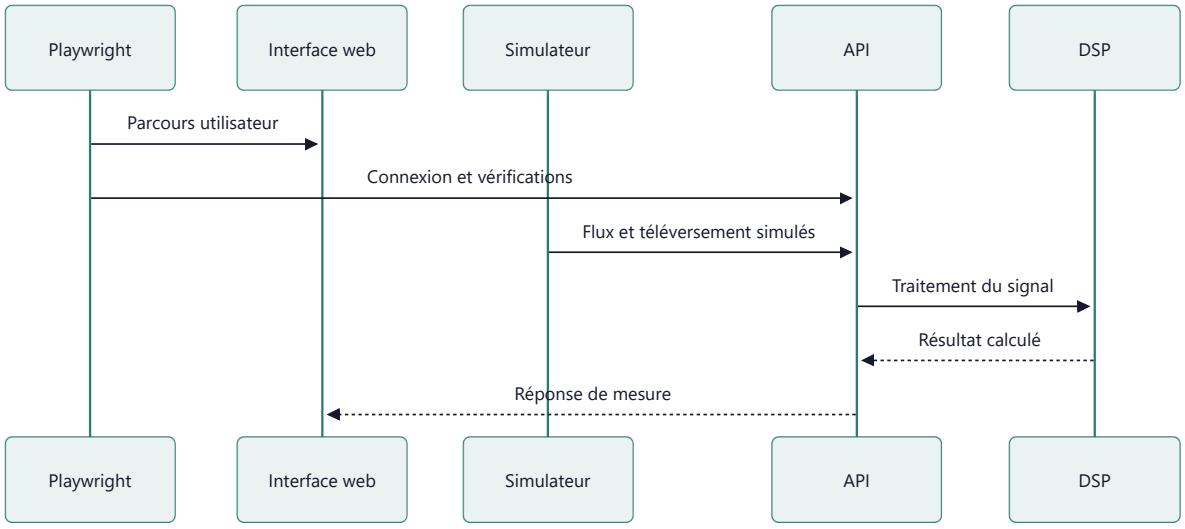
2. Objectifs

- Vérifier les huit étapes taguées et le parcours de banc (`e2e/tests/demo.spec.ts` ; `e2e/tests/bench.spec.ts`).
- Utiliser le simulateur avec la même interface de protocole que le dispositif (`scripts/simulator/src/dsfav_sim/`).
- Décrire clairement ce que la démonstration prouve et ce qu'elle ne prouve pas (AGENTS.md, §4 règle 10).

3. Démarche et méthode

Playwright pilote l'interface et appelle les routes API; le simulateur produit les événements et envoie les captures. Le test d'intégration de démonstration rapporte dix réussites; un run antérieur avait neuf réussites et un test ignoré faute de variable `WEB_URL` (journal du hub, T-s14).

4. Conception



Le scénario de test exécute le simulateur depuis le projet e2e et vérifie la réponse du téléversement (`e2e/tests/demo.spec.ts`).

5. Réalisation

Les étapes vérifient accès aux comptes de démonstration, capture normale, refus de pression basse puis récupération, mouvement léger, rejet de mouvement fort, répétition simulée, historique et avertissement, puis coupure de connexion. Ces intitulés sont définis dans `e2e/tests/demo.spec.ts`. Le scénario de banc se trouve dans `e2e/tests/bench.spec.ts`.

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Suite e2e de démonstration	Dix tests passés dans le run rapporté par l'orchestrateur	journal du hub, T-s14
Run initial du builder	Neuf tests passés, un ignoré car `WEB_URL` n'était pas défini	journal du hub, T-s14
Étape de coupure réseau	Le simulateur confirme la coupure et l'absence de téléversement brut	`e2e/tests/demo.spec.ts`, test `[NO-LOSS]`
Mise en file et reprise après perte	Non prouvées par cette étape; limite inscrite au backlog	`docs/BACKLOG.md`, entrée « Offline queue… »; journal du hub, T-s14

7. Décisions et justifications

La démonstration utilise les services de développement et des données synthétiques, sans faire de déclaration clinique (AGENTS.md, §4 règle 10). Les scripts d'authentification utilisent les comptes seedés du test, et les secrets de périphérique sont fournis par l'environnement du processus (e2e/tests/demo.spec.ts).

8. Limites et points ouverts

La preuve du scénario « drop » s'arrête avant l'envoi du fichier brut; elle ne démontre pas la reprise idempotente d'un élément mis en file (`e2e/tests/demo.spec.ts`; `docs/BACKLOG.md`). Le scénario d'historique dépend de la session web et des comptes seedés (e2e/tests/demo.spec.ts). La simulation ne valide ni capteur physique ni diagnostic.

9. Ce qu'il faut retenir

- La suite relie l'interface, l'API, le simulateur et le DSP.

- Les scénarios vérifient le comportement logiciel observable.
- La reprise après coupure reste une limite de la preuve actuelle.
- Aucun test synthétique ne prouve une efficacité clinique.

---

10. Questions possibles du jury

- **Que prouve le test e2e ?** Que le parcours logiciel fonctionne sur la pile et les données testées (journal du hub, T-s14).
- **La coupure réseau prouve-t-elle qu'aucune séance n'est perdue ?** Non; ce test vérifie l'arrêt avant téléversement, pas la reprise depuis une file (`e2e/tests/demo.spec.ts` ; `docs/BACKLOG.md`).
- **Le résultat démontre-t-il une capacité diagnostique ?** Non, il n'y a aucune conclusion clinique à partir de données synthétiques (AGENTS.md, §4 règle 10).

Références

`e2e/tests/demo.spec.ts` · `e2e/tests/bench.spec.ts` · `scripts/simulator/src/dsfav_sim/` · `.agent-hub/events.jsonl` (T-s14) · `docs/BACKLOG.md` · `pre-dev/10-build-plan.md`

PARTIE 3

Interfaces, sécurité, IA et matériel

Interfaces médicales, droits et authentification, IA explicable, protocole appareil et étude sur banc.

LOT DE TRAVAIL 15

Interfaces et graphiques médicaux

Jalon M5 (reprise) | terminé (finitions en cours) | 2026-11-09

Résumé

Un audit des écrans a montré que les interfaces ressemblaient à un prototype fait main : les bibliothèques de conception approuvées étaient installées mais presque inutilisées, 16 fichiers dessinaient leurs graphiques en SVG à la main et 100 valeurs de couleur étaient écrites en dur (sources en §6). Deux lots ont corrigé les parties qui portent des graphiques et du contenu médical : l'acquisition en direct, le suivi du patient et le spectrogramme. Le reste de l'interface n'a volontairement pas été remplacé.

1. Contexte et problématique

Le propriétaire du projet a constaté que la page d'acquisition n'affichait que des nombres à la place du graphique, et que la page patient paraissait « codée en dur ». Un audit (`docs/UI-AUDIT.md`) a listé les causes et les défauts page par page.

2. Objectifs

- Un graphique du signal en direct avec axes, unités, légende et état d'attente, même sans appareil.
- Un mode démonstration qui joue une acquisition complète enregistrée, étiquetée « Données simulées ».
- Des graphiques d'analyse (tendance des séances, spectrogramme, mini-courbes) lisibles et accessibles.

3. Démarche et méthode

Captures d'écran avant et après (`docs/visual/screens/`), revue de chaque livraison par comparaison visuelle et non seulement par les tests. Une première livraison a été rejetée parce que les tests passaient mais que la mise en page était inchangée ; une seconde a corrigé l'essentiel et neuf défauts résiduels ont fait l'objet d'une tâche de finition.

4. Conception

BESOIN	CHOIX
Signal en direct 1 kHz	uPlot (déjà installé) : axes 0 à 25 s, échelle symétrique en g, masque de mouvement au-delà de 0,08 g, bande de pression 3,0 à 8,5 kPa
Tendances, mini-courbes, barres	Recharts (déjà installé) : ligne de référence de la baseline, points colorés par statut avec texte
Spectrogramme	Dessin direct, sans dépendance, derrière des propriétés stables pour pouvoir passer plus tard à ECharts
Aucune bibliothèque médicale spécialisée adaptée n'existe (recherche : Medplum, Terra UI, composants React du NHS, Carbon) ; l'approche retenue est celle des applications de santé : bibliothèque générale et composants cliniques par-dessus.	

5. Réalisation

- Page d'acquisition : axes, grille, légende, message d'attente, bande de pression, mode démonstration.
- Page patient : graphique RMS par séance avec dates en français (l'API renvoie désormais la date de chaque mesure, changement de contrat), ligne de baseline, tableau de données accessible, liste des 10 dernières séances avec « Voir tout », virgule décimale française, spectrogramme avec axes en Hz et en secondes.
- Remplacement des graphiques dessinés à la main par des composants réutilisables ; zéro valeur de couleur en dur dans les graphiques et les fonctionnalités.

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Tests web	138 passent, 0 échec	`pnpm --filter @dsfav/web test`, 2026-11-09
Valeurs de couleur en dur dans graphiques et fonctionnalités	100 avant, 0 après	recherche par expression régulière dans `src/`
Fichiers dessinant du SVG à la main	16 avant l'audit	`docs/UI-AUDIT.md`
Captures	avant et après de la page patient et de la page d'acquisition	`docs/visual/screens/before-*.png`, `t17r1-patient.png`, `mid-capture-demo.png`

7. Décisions et justifications

- Pas de refonte complète de l'interface : seules les parties graphiques et médicales changent.
- Aucune dépendance nouvelle sans accord : ECharts reste une option soumise au propriétaire.
- Les nouvelles couleurs viennent des jetons de conception, jamais du code.

8. Limites et points ouverts

Neuf défauts visuels résiduels (badges qui passent à la ligne, mini-courbes trop exagérées, axes qui chevauchent le signal sur la page d'acquisition) sont confiés à une tâche de finition. Les pages devices, notes, historiques, journal d'audit et utilisateurs n'ont pas encore été capturées une par une. Les données affichées sont synthétiques.

9. Ce qu'il faut retenir

- Des tests qui passent ne prouvent pas qu'un écran est correct : il faut regarder les captures.
- Installer une bibliothèque ne suffit pas : encore faut-il l'utiliser.
- Un graphique médical se juge à ses axes, ses unités et sa lecture sans la couleur seule.

10. Questions possibles du jury

- **Pourquoi pas une bibliothèque médicale dédiée ?** Aucune ne couvre un signal vibratoire avec baseline personnelle ; les applications de santé assemblent des composants généraux.

- **Les graphiques sont-ils fiables cliniquement ?** Ils affichent correctement des données synthétiques ; aucune conclusion clinique n'en découle.
- **Que se passe-t-il sans l'appareil ?** Le graphique reste affiché avec « En attente de l'appareil », et le mode démonstration joue une acquisition simulée.

Références

`docs/UI-AUDIT.md` · `apps/web-nextjs/src/components/charts/` · `docs/visual/screens/` · `docs/DECISIONS.md` (D-010)

LOT DE TRAVAIL 16

Droits, authentification et durcissement

Jalon M6 / durcissement | terminé | 2026-11-09

Résumé

Une table de droits unique (rôle × page × action) est partagée par l'API et le web. Un test parcourt les 33 routes de l'application pour les 4 rôles. Pendant ce travail, un défaut réel a été trouvé et corrigé : deux routes d'administration répondaient à tout le monde, même sans connexion (sources en §6).

---

1. Contexte et problématique

Avant ce lot, seul le rôle « interne » était restreint côté web, alors que la matrice canonique (`pre-dev/09-canonical-spec.md` §10) distingue quatre rôles. Un contrôle d'accès incomplet est une faute de sécurité clinique : l'administrateur ne doit pas voir les données patient, l'infirmier ne doit pas publier la configuration.

---

2. Objectifs

- Une seule source de vérité pour les droits, lue par l'API et par le web.
- Les pages manquantes de chaque rôle : journal d'audit, utilisateurs, notes et suivis, historique des références, historique du banc, pages « accès refusé » et « introuvable ».
- Un test qui échoue si une route est oubliée ou non protégée.

3. Démarche et méthode

Tâche T-w13 (droits et pages), puis T-w10 (durcissement). Les parcours Playwright, un par rôle, ont révélé le défaut décrit plus bas que les tests unitaires n'avaient pas vu.

4. Conception

ACTION	INFIRMIER	NÉPHROLOGUE	INTERNE	ADMIN
Capturer	oui	non	non	non
File et historique	oui (lecture)	oui	non	non
Notes, suivi, re-référence	non	oui	non	non
Banc	non	lecture	oui	non
Appareils	voir	voir	non	calibrer
Configuration clinique	non	publier	non	brouillon
Journal d'audit	non	oui	non	oui
Source : `docs/AUTHZ.md`, copie de la matrice canonique.				

5. Réalisation

- Table partagée dans `apps/api-nestjs/src/common/authorization.ts`, utilisée par le garde de rôles de l'API et par le garde de routes du web.
- Déconnexion, bandeau d'expiration de session, menu de profil, redirection vers l'accueil du rôle.
- Durcissement : limites de taille de corps (16 Mo, 50 éléments par lot de synchronisation) avec le corps d'erreur du contrat, test de charge à 4 envois parallèles, analyse des données personnelles dans le schéma, le contrat, les jeux de test, les corps d'erreur et les journaux (`docs/SECURITY.md`).

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Matrice sur toutes les routes	33 routes × 4 rôles testées	`apps/api-nestjs/test/`, tâche T-w10
Suite API	143 passent, 0 échec, 8 ignorés	`pnpm --filter @dsfav/api test`, 2026-11-08
Parcours par rôle	4 sur 4 passent	Playwright `e2e/tests/roles.spec.ts`
Défaut trouvé	`GET /audit` et `GET /users` répondaient 200 sans jeton	observation du 2026-11-08, corrigée dans la foulée

7. Décisions et justifications

- La matrice canonique l'emporte sur le plan de travail : la page des notes est réservée au néphrologue, contrairement à ce que le plan prévoyait d'abord.
- Le défaut venait de contrôleurs qui déclaraient des rôles sans activer le garde ; le test qui parcourt toutes les routes empêche ce type d'oubli de revenir.
- Port de l'API fixé à 3101 (décision D-013) : le port 3001 est pris par un autre projet, ce qui avait fait échouer une connexion.

8. Limites et points ouverts

Aucune revue de sécurité externe ; pas de limitation de débit ni de verrouillage de compte ; les mots de passe de démonstration ne servent qu'au développement.

9. Ce qu'il faut retenir

- Un contrôle d'accès se teste route par route et rôle par rôle, pas en échantillon.
- Un test de parcours réel a trouvé ce que les tests unitaires ne voyaient pas.
- La règle de la spécification l'emporte sur l'idée du chef de projet.

10. Questions possibles du jury

- **Comment garantir qu'une nouvelle route est protégée ?** Le test parcourt toutes les routes de l'application et échoue si l'une répond sans jeton.
- **Pourquoi l'administrateur ne voit-il pas les patients ?** La matrice canonique lui réserve le parc d'appareils et la métrologie.
- **Y a-t-il des données personnelles ?** Non : seuls des pseudonymes existent, et un test le vérifie dans le schéma, le contrat, les fixtures et les journaux.

Références

`docs/AUTHZ.md` · `docs/SECURITY.md` · `docs/DECISIONS.md` (D-013) · `pre-dev/09-canonical-spec.md` §10

LOT DE TRAVAIL 17

Couche d'IA explicable

Jalon M2 / M5 (extension) | terminé (chemin vers les données réelles : voir §8) | 2026-11-09

Résumé

Le système n'utilisait aucun apprentissage : un statut vient de règles fixes comparant la séance à la baseline du patient. Une couche d'IA **explicable** a été ajoutée à côté, comme second avis optionnel. Les règles continuent de décider du statut ; l'IA ne déclenche jamais d'alerte, n'affiche rien pendant la constitution de la baseline et montre toujours ses preuves (sources en §6).

1. Contexte et problématique

Une IA « boîte noire » est incompatible avec une aide à la décision médicale, et sans données réelles étiquetées on ne peut rien entraîner de valide. Un modèle appris sur le simulateur apprendrait le simulateur.

2. Objectifs

- Un avis complémentaire transparent : mesure de la distance à la baseline du patient et classifieur simple.

- Aucune conséquence sur le statut des règles, aucune alerte propre.
- Un étiquetage honnête : « Données synthétiques, non cliniques » tant que l'entraînement est synthétique.

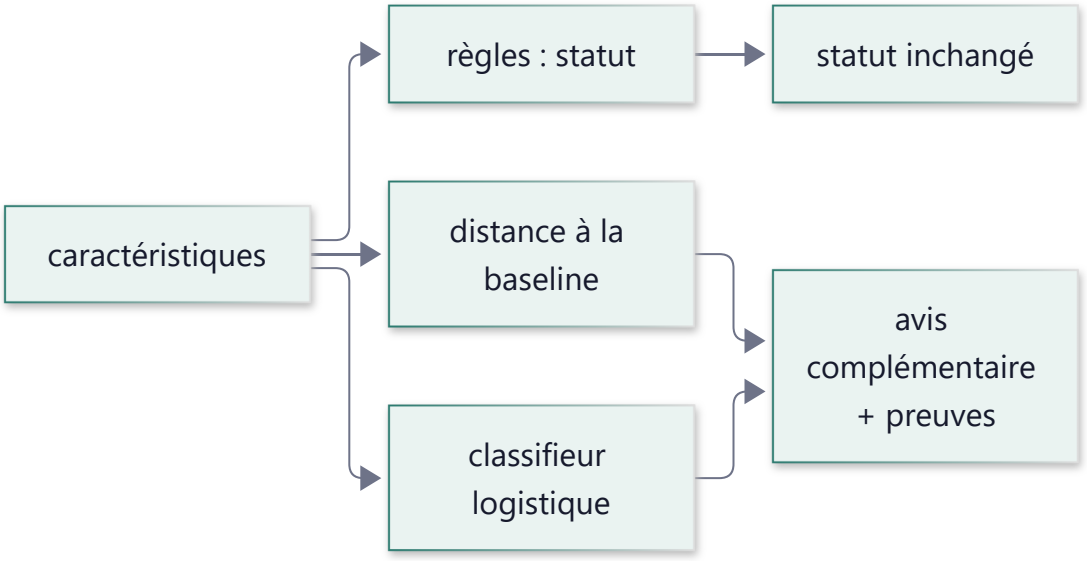
---

3. Démarche et méthode

Décision D-013 du plan (modèles transparents uniquement, `numpy` et `scipy` déjà installés, aucune dépendance nouvelle). Conception documentée dans `docs/ai/design.md`. Réalisation en deux passes : moteur côté DSP, puis contrat, API et carte d'interface.

---

4. Conception



Le résultat porte : les caractéristiques mesurées, les valeurs de référence, les principaux contributeurs, la version du modèle et l'étiquette des données.

---

5. Réalisation

- Modules `apps/dsp-python/src/dsfav_dsp/ai/` : caractéristiques robustes, normalité, classifieur entraînable de façon déterministe, fichier de modèle versionné avec ses métriques.
- Le pipeline renvoie un bloc `aiOpinion` optionnel, seulement si un fichier de modèle existe et si la baseline est verrouillée.
- Contrat : champ optionnel `aiOpinion`. API : stockage et restitution inchangés.
- Interface : carte « Avis complémentaire » (preuves d'abord, puis niveau), masquée si absente ou pendant `BASELINE_BUILDING`, avec le même avertissement unique que le reste de l'écran.

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Tests du moteur	38 passent (25 avant la couche)	`uv run pytest`, 2026-11-09
Statut des règles identique avec et sans IA	test dédié	`apps/dsp-python/tests/ai`
Aucune sortie IA pendant la constitution de la baseline	test dédié	idem
Fonctionnement sans fichier de modèle	test dédié	idem
Carte web : cas affichée, masquée, un seul avertissement	8 tests	`result-card-ai.test.tsx`, `patient-detail-ai.test.tsx`

7. Décisions et justifications

- Des modèles simples et lisibles plutôt qu'une boîte noire, car une explication chiffrée est exigée par les règles du projet.
- Aucun « score IA » sans la mesure à côté (règle 8 de `AGENTS.md`).
- Le fichier de modèle est optionnel : l'application fonctionne identiquement sans lui.

8. Limites et points ouverts

L'entraînement est **synthétique** : la couche démontre la méthode, pas une valeur clinique (règle 10). Le chemin vers les données réelles (export des caractéristiques étiquetées, réentraînement, validation, seuil d'acceptation à confirmer avec le superviseur) est documenté dans `docs/ai/real-data.md` et testé sur les données synthétiques de l'étude virtuelle. Le point d'accès d'historique du patient ne renvoie pas encore l'avis IA par séance.

9. Ce qu'il faut retenir

- L'IA ajoute un second avis lisible ; elle ne remplace pas les règles.
- Sans données réelles étiquetées, on ne prétend pas à une performance clinique.
- Un modèle absent ne doit rien casser.

10. Questions possibles du jury

- **Pourquoi pas un réseau de neurones ?** Il faudrait des milliers de séances réelles étiquetées, et son explication serait faible.
- **L'IA peut-elle déclencher une alerte ?** Non, jamais : seules les règles configurées produisent un statut.
- **Comment passer aux données réelles ?** Étude sur banc, export étiqueté, réentraînement, validation sur un jeu séparé, puis décision du superviseur.

Références

`docs/ai/design.md` · `docs/ai/real-data.md` · `apps/dsp-python/src/dsfav_dsp/ai/` · `docs/DECISIONS.md` · `AGENTS.md` §4 règles 8 et 10

LOT DE TRAVAIL 18

Protocole appareil et squelette de firmware

Jalon H3 (préparation) | terminé sur PC ; jamais exécuté sur une carte | 2026-11-09

Résumé

Un squelette de firmware pour l'ESP32-C3 construit les trames exactement comme le protocole écrit, vérifiées octet par octet contre l'encodeur du simulateur, avec un tampon circulaire hors ligne qui n'écrase jamais des données non synchronisées. **Rien n'a été exécuté sur une vraie carte** : les composants ne sont pas commandés (sources en §6).

1. Contexte et problématique

Le projet dépend d'un appareil réel, mais le matériel n'est pas encore acheté. Pour ne pas perdre de temps, le code qui peut être prouvé sur un PC (formats, mémoire tampon) est écrit et testé maintenant ; les étapes qui exigent le matériel restent une liste de contrôle.

2. Objectifs

- Des trames identiques à celles du simulateur et du protocole (`docs/protocol/device-protocol.md`).

- Une mémoire tampon hors ligne sans perte (règle « ne jamais perdre une séance »).
- Aucun identifiant Wi-Fi ni jeton dans le code source.

3. Démarche et méthode

Tests natifs sur PC compilés avec le compilateur `zig` trouvé sur la machine (pas de chaîne PlatformIO nécessaire pour la partie testée). Une première livraison a été rejetée par la revue parce qu'elle ne produisait que l'en-tête de 18 octets : la seconde emballe aussi les échantillons.

4. Conception

Format brut : en-tête (magie `DSFV`, version, fréquences, nombres d'échantillons) puis échantillons ADXL355 et LIS2DW12 sur 16 bits en petit-boutiste, et pression en centi-kPa. Le tampon circulaire refuse l'écriture plutôt que d'écraser un élément non synchronisé.

5. Réalisation

Dossier `firmware/esp32c3-dsfav/` : bibliothèque de trames, tampon, tests natifs, ébauches d'acquisition (ADXL355, LIS2DW12, capteur de pression, client WebSocket). Liste de mise en route `docs/hardware/bringup.md` pour les étapes H0 à H3.

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Trame complète identique à l'encodeur du simulateur	oui	`node test/run-native.mjs`, étiquette `[G1]`
Tampon : refus d'écraser un élément non synchronisé	oui	étiquette `[NO-LOSS]`
Aucun identifiant ou jeton dans `src` et `lib`	oui	contrôle par recherche dans le même exécuteur
Exécution sur une carte	**non réalisée**	matériel non commandé

7. Décisions et justifications

- Les identifiants Wi-Fi viennent d'une phase de mise en service, jamais du code (règle 6 de `AGENTS.md` §5).
- Le compilateur local sert aux tests ; la compilation pour la carte est une étape distincte à valider avec la chaîne PlatformIO.

8. Limites et points ouverts

Le bruit réel des capteurs, l'étalonnage de la pression, la consommation, la portée Wi-Fi et la tenue du tampon sur la mémoire flash réelle ne sont pas testés. La taille de la flash de la carte reste une question ouverte qui fixe la taille du tampon. L'ébauche de la carte n'est pas encore branchée sur la bibliothèque de trames.

9. Ce qu'il faut retenir

- On prouve sur PC tout ce qui ne dépend pas du matériel, et on dit clairement le reste.
- Une trame se vérifie contre une autre implémentation, pas contre elle-même.
- La mémoire tampon est une exigence de sécurité des données, pas un détail.

10. Questions possibles du jury

- **Le firmware fonctionne-t-il ?** Ses formats et son tampon sont prouvés sur PC ; son exécution sur carte est la prochaine étape.
- **Que se passe-t-il quand le Wi-Fi tombe ?** Les séances restent dans le tampon et sont envoyées plus tard, une seule fois grâce à la clé d'idempotence.
- **Pourquoi ne pas commander le matériel d'abord ?** Cela reste nécessaire ; ce travail réduit simplement le risque le jour où les pièces arrivent.

Références

`firmware/esp32c3-dsfav/` · `docs/protocol/device-protocol.md` · `docs/hardware/bringup.md` · `scripts/simulator/`

LOT DE TRAVAIL 19

Kit d'étude sur banc et étude virtuelle

Jalon M9 (préparation) | terminé (étude virtuelle réalisée sur données synthétiques) | 2026-11-09

Résumé

Tout ce qu'il faut pour réaliser l'étude sur banc le jour où un appareil existe est prêt : protocole écrit, fiches de données, import de résultats, rapport de concordance et outil de proposition de seuils soumis à approbation. Une étude **virtuelle** sur le simulateur, étiquetée synthétique, démontre la méthode de bout en bout. Elle ne prouve rien de clinique (sources en §6).

1. Contexte et problématique

Les seuils cliniques sont des valeurs par défaut « à calibrer au banc ». Sans appareil, on ne peut pas mesurer : on prépare donc la méthode, les formulaires et les outils d'analyse pour que l'étude réelle soit rapide et sans improvisation.

2. Objectifs

- Un protocole sur fantôme (aucun humain), valeurs de pince de 0, 30, 50 et 75 %.
- Un import de données qui refuse toute colonne de données personnelles.
- Un rapport de concordance entre la palpation et la mesure objective.
- Une proposition de seuils qui ne modifie jamais la configuration sans approbation du néphrologue et sans événement d'audit.

3. Démarche et méthode

Tâche T-w09 pour le kit. La règle du banc est imposée par le serveur : aucune donnée objective avant l'enregistrement de la réponse de palpation (règle 9 de `AGENTS.md`).

4. Conception



5. Réalisation

- Documents : `docs/bench-study/protocol.md`, `data-sheet.md`, `import-format.md`.
- API : import de résultats par fichier CSV, rapport de concordance, propositions de seuils que seul un néphrologue différent de l'auteur peut approuver dans une transaction auditée.
- Étude virtuelle : exécution automatique de 8 séances (2 par niveau de pince : 0, 30, 50 et 75 %) sur fantôme virtuel, via l'API réelle, avec un rapport en français (`docs/bench-study/virtual-study-report.md`) et un fichier de caractéristiques étiquetées (`apps/dsp-python/data/virtual-bench-features.csv`) pour le réentraînement de la couche d'IA. Les réponses de palpation sont scriptées : la concordance affichée (environ 92 %) est obtenue par construction et ne mesure aucune perception humaine.

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Tests du module banc et de la configuration	20 passent	`pnpm --filter @dsfav/api test`, 2026-11-08
Rejet d'un fichier avec colonne de données personnelles	testé	module banc, test d'import
Étude virtuelle	8 séances, concordance moyenne 92,19 % (palpation scriptée, donc par construction)	`docs/bench-study/virtual-study-report.md`
Chemin de réentraînement de l'IA	43 tests du moteur passent, dont 4 de réentraînement (rejet d'une colonne de données personnelles compris)	`uv run pytest`, 2026-11-10

7. Décisions et justifications

- Étude sur fantôme uniquement : aucun comité d'éthique n'est nécessaire pour cette partie ; toute donnée humaine ultérieure en exigerait un.
- Les jeux de données synthétiques portent l'étiquette « Données synthétiques, non cliniques ».

8. Limites et points ouverts

Aucune donnée réelle. Les statistiques de concordance sont élémentaires et ne remplacent pas un plan d'analyse validé par le superviseur. Le fantôme et la pompe restent à réunir.

9. Ce qu'il faut retenir

- Préparer la méthode avant le matériel réduit le risque de l'étude réelle.
- Une proposition de seuils ne s'applique jamais sans approbation et trace d'audit.
- Un rapport sur données synthétiques démontre une méthode, jamais une performance.

10. Questions possibles du jury

- **Pourquoi quatre niveaux de pince ?** Ils correspondent à la sévérité simulée de la sténose du fantôme (0, 30, 50, 75 %).
- **Qui valide les nouveaux seuils ?** Un néphrologue différent de l'auteur de la proposition, avec un événement d'audit.
- **L'étude virtuelle prouve-t-elle quelque chose ?** Elle prouve que la chaîne d'analyse fonctionne ; la validité clinique exige l'étude réelle.

Références

`docs/bench-study/` · `apps/api-nestjs/src/modules/bench/` · `apps/api-nestjs/src/modules/config/` · `AGENTS.md` §4 règles 9 et 10

PARTIE 4

Reprise des écrans et synthèse

Intégration, limites, refonte des écrans par les compétences de conception.

LOT DE TRAVAIL 20

Intégration, limites et suite

Jalon synthèse | terminé (chiffres à jour au 2026-11-10) | 2026-11-09

Résumé

Le projet est un système logiciel complet et testé pour un **appareil virtuel** : simulateur, API, traitement du signal, interface par rôle, banc de formation, synchronisation sans perte et couche d'IA explicable. Il n'est pas prêt pour le matériel réel ni pour un usage clinique : les composants ne sont pas commandés, les seuils ne sont pas étalonnés et toutes les données sont synthétiques (sources en §6).

1. Contexte et problématique

Après le jalon du 2026-11-07, un plan de 30 jours a été condensé en une journée de travail avec plusieurs agents : fermeture des lacunes de contrat, résilience hors ligne, droits et pages par rôle, écrans de flotte et de suivi, firmware, kit de banc, durcissement, IA explicable, graphiques médicaux.

2. Objectifs

Dire honnêtement où en est le projet, avec des chiffres vérifiables, ce qui est prouvé, ce qui ne l'est pas et la suite.

3. Démarche et méthode

Chaque tâche : test d'abord, revue indépendante, vérification par l'orchestrateur, commit. Quand les agents externes n'étaient plus disponibles, l'orchestrateur a assuré la revue avec l'accord explicite du propriétaire, et la note de chaque acceptation le dit.

4. Conception

Chaîne : simulateur ou appareil → API (WebSocket et envoi brut) → service de traitement du signal → base de données → interface web par rôle. Documentation : `docs/ARCHITECTURE.md`, `docs/GUIDE-PAGES.md`.

5. Réalisation

DOMAINE	ÉTAT
Contrat et données	20 chemins, route des notes, champs de synchronisation, date de chaque mesure
Traitement du signal	règles G1 à G9, modèle de signal paramétrique, couche d'IA
Interface	rôles, accès, graphiques du signal et de la tendance, spectrogramme
Synchronisation	idempotente, sans perte, reprise après session expirée
Banc	formation, révélation côté serveur, kit d'étude
Matériel	squelette et liste de contrôle seulement

6. Vérification et résultats

SUITE	RÉSULTAT	COMMANDE, DATE
API (base de données neuve)	144 passent, 8 ignorés, 23 suites	`pnpm --filter @dsfav/api test`, 2026-11-10
Web	147 passent, 24 fichiers	`pnpm --filter @dsfav/web test`, 2026-11-10
Traitement du signal	43 passent	`uv run pytest`, 2026-11-10
Simulateur	30 passent	`uv run pytest`, 2026-11-10
Parcours de bout en bout	19 passent (démonstration en 8 étapes, hors ligne, rôles, accessibilité, captures)	`pnpm e2e`, 2026-11-10
Scripts de démonstration	4 passent	`node --test scripts/demo`, 2026-11-10
Firmware natif	trames, absence d'identifiants, tampon sans perte	`test/run-native.mjs`, 2026-11-10
Contrat d'API	valide	`pnpm contract`, 2026-11-10

7. Décisions et justifications

Voir `docs/DECISIONS.md` (D-001 à D-013). Les plus structurantes : thresholds en configuration, pas de dépendance sans accord, modèles transparents, port 3101.

8. Limites et points ouverts

- **Aucune validité clinique** : données synthétiques, seuils par défaut.
- **Matériel** : non commandé, firmware jamais exécuté sur une carte.
- **Étude sur banc** : kit prêt, aucune donnée réelle.
- **Production** : pas de déploiement, pas de revue de sécurité externe, pas de limitation de débit.
- **Interface** : la page des appareils affiche « seuils indisponibles » car la configuration ne contient pas de seuils de batterie ni de gain ; des dates brutes apparaissent encore sur la page des notes.
- **Backlog** (`docs/BACKLOG.md`) : liste des éléments en échec de synchronisation non affichée, historique des versions de configuration et brouillons d'administration sans API, bande de dispersion de la baseline, avis IA dans l'historique du patient.

9. Ce qu'il faut retenir

- Le logiciel et sa méthode sont prêts ; la preuve médicale ne l'est pas.
- Chaque chiffre de ce dossier renvoie à une commande ou à un fichier.
- La suite dépend d'une action humaine : commander le matériel, réaliser l'étude sur banc.

10. Questions possibles du jury

- **Peut-on l'utiliser sur des patients ?** Non : aucune validation clinique, aucune certification.
- **Que reste-t-il avant un prototype réel ?** Commander les composants, mettre en service les capteurs, relier le firmware à la bibliothèque de trames, étalonner la pression.
- **Quand les seuils seront-ils fiables ?** Après l'étude sur banc, avec approbation du superviseur.

Références

`docs/BACKLOG.md` · `docs/DECISIONS.md` · `docs/ARCHITECTURE.md` · `docs/GUIDE-PAGES.md` · `docs/PLAN-30-JOURS.md`

LOT DE TRAVAIL 21

Refonte des écrans avec les compétences de conception, et vérification dans un vrai navigateur

Jalon M5 (reprise 2) | terminé (finitions en §8) | 2026-11-10

Résumé

Après la livraison de LT-15, le propriétaire a relevé page par page des défauts d'usage : une page patient de 2000 px pour 2 séances, une page d'acquisition dont le bouton de départ était hors écran, des journaux illisibles (codes bruts, dates ISO), un profil utilisateur accessible seulement en bas de page. Neuf écrans ont été repris, un composant interactif a été créé (schéma de la fistule) et une règle de méthode a été ajoutée au dépôt : tout travail d'interface passe par des compétences de conception (`unslop`, `impeccable`, `hallmark`) et se prouve par une capture d'écran réelle. Résultat chiffré principal : la page patient passe de 2000 à 1186 px de haut (mesure Playwright, §6) et la suite web de 149 à 186 tests réussis (§6).

---

1. Contexte et problématique

LT-15 avait corrigé les graphiques de l'acquisition et du suivi patient. Il restait les autres écrans, construits au fil des lots à partir de composants voisins, sans passe de conception : tableau de bord, notes, historiques, paramètres, journal d'audit, appareils. Les cas d'usage concernés sont UC1 (acquisition), UC2 (pression), UC8 et UC9 (suivi et écart), UC11 (banc), UC13 (audit) du cahier des charges (`pre-dev/10-build-plan.md` §4). Une question du propriétaire a aussi mis en évidence un manque de méthode : les compétences de conception du projet n'avaient pas été chargées avant de construire ces écrans.

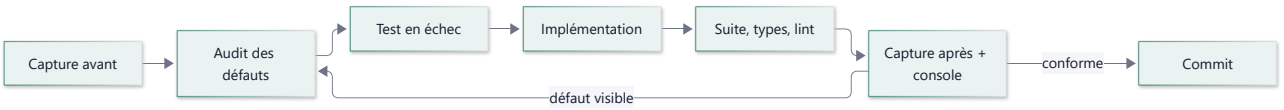
2. Objectifs

OBJECTIF	CRITÈRE D'ACCEPTATION
Aucune donnée machine à l'écran (dates ISO, codes d'énumération, identifiants longs)	Tests `[UC13]`, `[UC8]` sur les libellés et les dates; capture réelle
Page patient courte et lisible	Hauteur de page mesurée au navigateur à 1440 x 900
Action principale toujours accessible	Bouton Démarrer visible sans défilement à 900 px de haut
Profil utilisateur toujours visible et cliquable, y compris barre réduite	Test `[AUTHZ-MATRIX]` et capture aux deux largeurs
Règle de méthode durable	Règle inscrite dans `AGENTS.md`, `CLAUDE.md`, une compétence de projet et le générateur de briefs

3. Démarche et méthode

Pour chaque écran : (1) capture « avant » dans un navigateur piloté (Playwright), (2) audit des défauts, (3) test écrit avant le code et constaté en échec, (4) implémentation, (5) suite complète, vérification de types et lint, (6) capture « après », regard sur l'image, (7) commit. L'étape 6 n'est pas remplaçable par les tests : plusieurs défauts de ce lot n'ont été vus que sur la capture (§5, tableau des défauts).

À partir du commit `6eb420d`, les travaux d'interface suivent les compétences `unslop` (audit des signes d'interface générique, passes ordonnées), `impeccable` (registre « produit », lois de conception) et `hallmark` (audit et autocritique). La compétence `design-taste-frontend` est réservée aux pages de marque : son en-tête exclut les tableaux de bord et l'interface produit.



4. Conception

Trois choix structurants.

Un schéma interactif plutôt qu'une image. La page patient affichait deux fichiers SVG statiques. Le schéma est maintenant dessiné en SVG intégré (`apps/web-nextjs/src/features/patients/ui/fistula-map.tsx`). Quatre zones (anastomose, veine de drainage, site du capteur, artère d'apport) commandent un même panneau de détail. Les boutons radio sous le schéma sont le contrôle accessible (clavier, lecteur d'écran); les zones du dessin sont un raccourci à la souris vers le même choix. L'artère est dessinée pour le repère et le panneau indique « Non mesurée » : le capteur ne mesure que le thrill veineux.

Des onglets pour réduire la longueur. Tendance, spectrogramme, séances et notes sont regroupés (`patient-detail.tsx`). Les panneaux restent montés et masqués (`forceMount`) pour que les graphiques gardent leur taille. Les onglets sont posés sur la page et non dans une carte, pour éviter des cartes imbriquées.

Un composant de profil unique. `user-menu.tsx` remplace le texte « Déconnexion » masqué sous 1024 px : un seul bouton à toutes les largeurs, qui ouvre un menu (rôle, identifiant pseudonyme, déconnexion). Le menu se ferme avec Échap ou un clic extérieur et rend le focus au bouton.

5. Réalisation

ÉCRAN	CHANGEMENT PRINCIPAL	COMMIT
Tableau de bord	Cinq tuiles de comptage par statut qui filtrent la file; les séances de banc (sans patient) ne figurent plus dans la file	`b564365`
Acquisition	Liste réelle des patients et appareil enregistré lus par l'API, état hors ligne de l'appareil, case à cocher de protocole honnête, pression bloquée dite une seule fois	`455d2e5` , `8aaabfd`
Acquisition (finitions)	Barre départ/arrêt collée en bas, minuteur et barre de progression sur une ligne (`role="progressbar"`), carte de protocole sur surface neutre	`b1cabdf`
Courbe du signal	Une seule trace, couleurs lues dans le thème, grille alignée	`f418ca1`
Notes	Espace à deux panneaux : liste des patients, saisie de note, historique filtrable	`f418ca1` , `9b01c16`
Journal d'audit, références, banc, paramètres, appareils	Événements en français avec catégories, détails lisibles, cartes d'historique, seuils regroupés par thème	`19697e7`
Barre latérale	Fixée à la hauteur de la fenêtre, profil cliquable	`e9e68c2`
Page patient	Schéma interactif, onglets	`5e75ac4`

Composants créés : `fistula-map.tsx` , `user-menu.tsx` , `components/shared/patient-nav.tsx` , `notes-composer.tsx` , `notes-timeline.tsx` , `admin/audit-labels.ts` , `libs/format/datetime.ts` , `capture/data/capture-sources.ts` .

Défauts trouvés par la vérification dans le navigateur et non par les tests

DÉFAUT	CAUSE	CORRECTION
Liste de patients transparente	Deux classes CSS collées par erreur (`w-80bg-surface`) lors d'une modification	`f418ca1`
Courbe dédoublée, trait noir	Deux moteurs dessinaient les mêmes traces; un canvas ne résout pas les variables CSS	`f418ca1`
Erreur console sur 4 routes	Lecture de données hors d'une frontière `Suspense`	`f418ca1` , `9b01c16` , `19697e7`
Barre collée sans effet	`overflow-x-hidden` sur `<main>` crée un conteneur de défilement	`b1cabdf` (`overflow-x-clip`)
Avertissement de clé React dupliquée	Deux composants frères avec la même clé	`9b01c16`

6. Vérification et résultats

CONTRÔLE	RÉSULTAT	SOURCE
Suite web	186 tests réussis, 32 fichiers (149 tests, 25 fichiers au début du lot)	`pnpm exec vitest run` dans `apps/web-nextjs` , sortie « Test Files 32 passed, Tests 186 passed »
Types et lint	Aucune erreur	`pnpm exec tsc --noEmit` , `pnpm exec eslint`
Hauteur de la page patient	2000 px avant, 1186 px après (1440 x 900)	`document.documentElement.scrollHeight` ; `docs/visual/screens/before-patient-4471.png` , `after-patient-4471.png`
Bouton Démarrer	Environ 1000 px avant (hors écran), visible dans 900 px après	`before-capture2.png` , `after-capture2.png`
Console navigateur	0 erreur sur l'acquisition en mode démonstration	`after-capture2-demo.png`
Profil	Barre à 0 px du haut et bouton à 684 px sur 700 après défilement de 268 px	mesure `getBoundingClientRect` , `sidebar-wide-menu.png` , `sidebar-collapsed-menu.png`

La suite API (145 tests) n'a pas été relancée pour ce lot, hormis le test ajouté pour la file (`apps/api-nestjs/test/measurements.e2e-spec.ts` , `[UC8] [UC10]`) : non mesuré à nouveau sur l'ensemble.

7. Décisions et justifications

D-014 (`docs/DECISIONS.md`) : les compétences de conception modernes sont obligatoires pour tout travail d'interface, via `.claude/skills/dsfav-design/SKILL.md` . Le générateur de briefs (`tools/briefs/gen.mjs`) ajoute `dsfav-design` , `unslop` , `impeccable` et `hallmark` aux tâches web, et le noyau les copie dans le

cache des travailleurs (`.agent-hub/tools/skills.mjs` , racines `~/.claude/skills`). Le rapport de fin de tâche cite une règle appliquée par compétence.

---

8. Limites et points ouverts

- **Données de la page Appareils.** Les lignes sont réelles (table `Device` , lues par l'API à chaque chargement, `cache: no-store`) mais leurs valeurs viennent du jeu de départ : batterie 95, 78, 54 et 12 %, gain 1,039 pour UNIT-04 (`apps/api-nestjs/prisma/seed.ts:78-81`). La batterie et la date de synchronisation ne changent que lorsqu'un appareil appelle la synchronisation (`devices.service.ts:212-217`). Les 8 éléments en échec de UNIT-01 sont des restes de tests négatifs : 6 `PATIENT_UNKNOWN` , 1 `INVALID_HEADER` , 1 `SYNC_ITEM_FAILED` (requête SQL sur `SyncQueueItem` , 2026-11-10). La page n'est pas en temps réel : aucun rafraîchissement automatique.
- **Compétences.** Un audit `hallmark` n'a pas été exécuté; `impeccable document` n'a pas produit de `DESIGN.md` .
- **Tuiles de métriques** de la page patient : les mini-courbes sont plates avec 2 séances et n'apportent pas d'information.
- **Acquisition** : trois cartes de même taille en tête, pression affichée dans la carte et sous la courbe; à 900 px de haut la barre collée masque le bas du graphique jusqu'au défilement.
- **Aucune conclusion clinique** : les statuts affichés (par exemple « Évolution inhabituelle ») proviennent de signaux simulés et ne prouvent que le calcul (AGENTS.md §4 règle 10).
- Les captures d'écran ne remplacent pas un test d'utilisabilité avec des infirmiers ou des néphrologues : non mesuré.

---

9. Ce qu'il faut retenir

1. Un test unitaire vert ne prouve pas qu'un écran est correct : cinq défauts de ce lot n'ont été vus que dans un navigateur.
2. Une règle de méthode n'existe que si elle est écrite là où l'agent la lit (consignes du dépôt, compétence, générateur de briefs).
3. Une donnée « réelle » peut venir d'une base sans être vivante : il faut distinguer lecture réelle, valeur de départ et temps réel.
4. Un schéma interactif doit dire ce qu'il ne mesure pas (l'artère).
5. Réduire la longueur d'une page passe par le regroupement et non par la suppression d'information.

10. Questions possibles du jury

1. **Les valeurs de la page Appareils sont-elles temps réel ?** Non. Elles sont lues en base à chaque chargement; batterie et synchronisation ne bougent que par un appel de synchronisation (`devices.service.ts:212-217`).
2. **Pourquoi des captures d'écran en plus des tests ?** Cinq défauts visibles n'avaient aucun test en échec (§5, tableau).
3. **Que représente le schéma interactif ?** Quatre zones; trois portent une valeur réelle de la dernière séance valide, l'artère est indiquée « non mesurée » (`fistula-map.tsx`).
4. **Pourquoi `design-taste-frontend` n'est-il pas appliqué ?** Son en-tête l'exclut pour les tableaux de bord et l'interface produit (fichier de la compétence).
5. **Le statut « Évolution inhabituelle » a-t-il une valeur clinique ici ?** Non, les signaux sont simulés (AGENTS.md §4 règle 10; `docs/academic/ce-qui-est-prouve.md`).

Références

- Commits `b564365` à `b1cabdf` (`git log --oneline`).
- `docs/DECISIONS.md` D-014; `AGENTS.md` §5 règle 9; `.claude/skills/dsfav-design/SKILL.md` .
- `docs/visual/screens/` (captures avant et après), `docs/UI-AUDIT.md` , `docs/BACKLOG.md` .
- LT-15 (interfaces et graphiques médicaux), LT-16 (droits et sécurité), LT-20 (intégration et limites).

ANNEXE

Ce qui est prouvé, et ce qui ne l'est pas

à jour 2026-11-10

Prouvé par les tests logiciels

- Des cas synthétiques ciblés vérifient le calcul DSP et ses portes qualité; le compte rendu rapporte 25 tests réussis (journal du hub, T-s04; `apps/dsp-python/tests/`).
- Des tests web vérifient les états affichés, les routes et les composants; un run final rapporte 98 tests réussis (journal du hub, T-s20).
- La démonstration Playwright relie les services sur la pile active; le run rapporté a dix tests réussis (journal du hub, T-s14).

- Après la refonte des écrans (LT-21), la suite web rapporte 186 tests réussis sur 32 fichiers, avec types et lint sans erreur (`pnpm exec vitest run` dans `apps/web-nextjs` , 2026-11-10); les défauts visuels sont vérifiés par captures dans un navigateur et non par les seuls tests (LT-21 §5).
- Le serveur protège la révélation du banc et une logique de concordance est testée (`apps/api-nestjs/src/modules/bench/` ; journal du hub, T-s12).

Limites de preuve

- Les tests DSP et les scénarios de démonstration utilisent des données synthétiques. Ils prouvent le comportement des calculs dans les cas construits, pas une valeur clinique (AGENTS.md, §4 règle 10; `pre-dev/10-build-plan.md` , §1 principe 4).
- Les valeurs de profils et seuils de banc sont des placeholders à calibrer (`apps/api-nestjs/src/modules/bench/bench.constants.ts`).
- Plusieurs tests d'intégration API n'ont pas démarré dans les runs des builders parce que Jest ne résolvait pas `@prisma/adapter-pg` (journal du hub, T-s07, T-s11, T-s12, T-s19).
- Le test de coupure du simulateur ne prouve pas la mise en file puis la reprise du téléversement (`e2e/tests/demo.spec.ts` ; `docs/BACKLOG.md`).
- Les valeurs de la page Appareils (batterie, gain, dernière synchronisation, éléments en échec) viennent du jeu de départ et de tests négatifs; la page n'est pas en temps réel (LT-21 §8; `apps/api-nestjs/prisma/seed.ts:78-81`).

Critères avant toute conclusion clinique

Une étude sur banc avec matériel mesuré doit calibrer les seuils; la spécification indique que les valeurs ouvertes sont des valeurs par défaut à calibrer au jalon M9 (`pre-dev/10-build-plan.md` , §1 principe 6; §3 M9). Toute décision d'examen reste médicale (AGENTS.md, §4 règle 1).